Optimizing Query Decomposition for Reliable Validation

Shuo Zhang
Duke University
Durham, North Carolina, USA
shuo.zhang2@duke.edu

Sudeepa Roy
Duke University
Durham, North Carolina, USA
sudeepa@cs.duke.edu

Jun Yang
Duke University
Durham, North Carolina, USA
junyang@cs.duke.edu

ABSTRACT

Large language models (LLMs) have made querying databases more accessible, but they cannot guarantee that the SQL they generate is correct. Without a trusted reference query, a validator (a human or an LLM) must determine whether a generated query answers the intended question. Enterprise questions often require queries with multiple joins, nested subqueries, and several levels of aggregation, and validating such a query in a single judgment is error-prone. Decomposition can simplify this task by rewriting a query into smaller queries, or blocks, that recompose to the original, each checked against its own sub-question.

Decompositions differ, however: smaller blocks make each judgment easier but require more judgments, and the best balance depends on the validator's ability. We formulate *validation-aware query decomposition* as an optimization problem. Just as a query optimizer chooses among equivalent plans by estimated execution cost, we choose among decompositions by how reliably a given validator is expected to check them. We model how task difficulty and validator ability jointly affect validation reliability and define an objective that balances reliability against the number of required judgments. Over a space of decompositions derived from logical-plan rewrites, our search finds an optimal one. We implement this approach in DecoVal, a validation pipeline that presents each block with its sub-question and input interface, combines local verdicts into a query-level decision, and localizes errors to individual blocks.

A user study shows that decomposition helps human validators catch and locate errors: with DecoVal, participants caught 91% of the errors, up from 36% without decomposition, and located the error in nearly every rejection. Experiments on BEAVER and Spider 2.0 with LLM validators show that DecoVal also increases error-detection recall over whole-query validation while maintaining high precision, and that the best granularity depends on the validator.

PVLDB Reference Format:
Shuo Zhang, Sudeepa Roy, and Jun Yang. Optimizing Query Decomposition for Reliable Validation. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Inverse0/validation-aware-decomposition.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

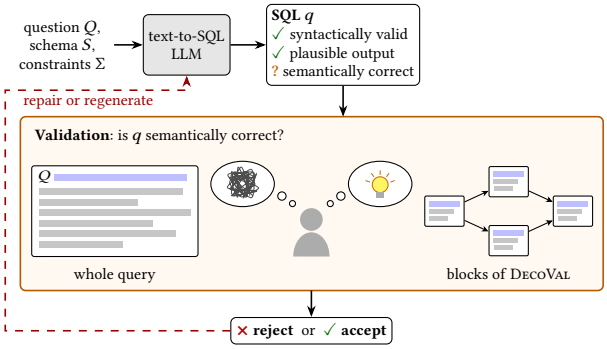


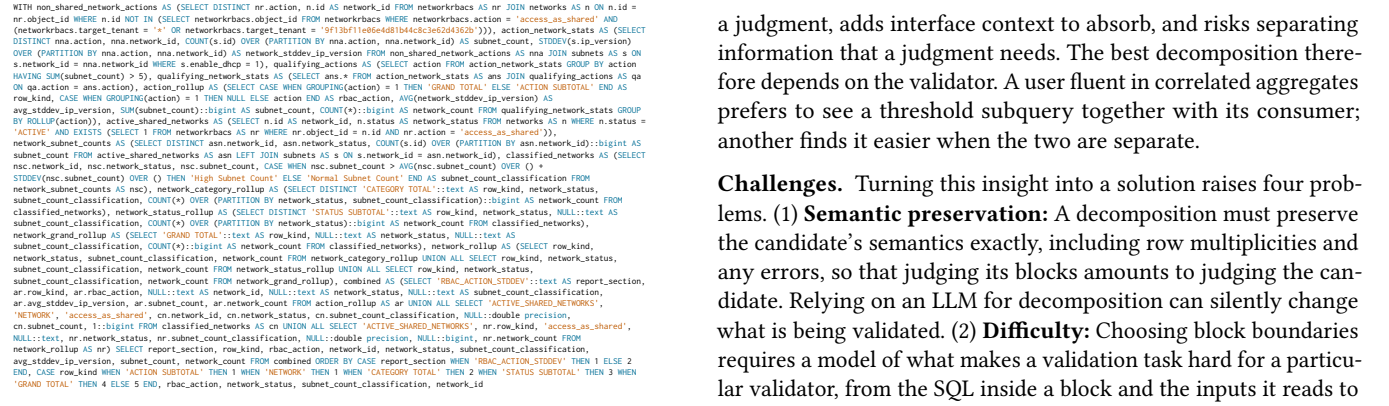
Figure 1: Validating LLM-generated SQL. The generated query q can be valid and return plausible output, but a validator needs to decide whether it is semantically correct.

1 INTRODUCTION

The rapid growth of AI-assisted querying is changing who can obtain value from data [15]. A user with limited SQL experience can now describe a complex business question in natural language and receive sophisticated SQL within seconds from an LLM [16, 25, 31]. But how can a user know whether the generated SQL is correct? A query can be syntactically valid and return plausible results but still return a semantically wrong answer; a user who trusts it acts on wrong information, which could lead to serious consequences [3]. The bottleneck has thus shifted rather than disappeared: the central question is no longer only whether a model can produce a query, but whether the produced query implements the intended analysis.

In this work, we address the problem of *Reference-Free Validation* (Figure 1): given a natural-language question Q, a database schema S with constraints Σ (e.g., keys and foreign keys), and a candidate SQL query q over S, we want to determine whether q correctly answers Q. When a trusted reference implementation query for Q exists, assurance reduces to equivalence checking, for which provers and bounded checkers are available [24, 81]. In practice, such a reference rarely exists: writing it is the very task the user delegated to the LLM. Executing the candidate cannot settle the question: enterprise data is often private [9], and an execution only allows checking on one database instance, where subtle errors, such as mishandling duplicates, `NULL`s, or missing joins, may not surface. Without a reference, the specification is the request itself, and the decision rests with a *validator*—a human user or an LLM—who reads the candidate and judges whether it answers the question.

One reason why this judgment is hard is that it is monolithic: the validator must reason with the whole query including every join, filter, aggregation, nested structure, and correlation at once. Figure 2 shows a query that an LLM generated for a question of the BEAVER enterprise benchmark (neutron_152, [9]). It chains



```
WITH non_shared_network_actions AS (SELECT DISTINCT nr.action, n.id AS network_id FROM networkrbacs AS nr JOIN networks AS n ON n.id =
nr.object_id WHERE n.id NOT IN (SELECT networkrbacs.object_id FROM networkrbacs WHERE networkrbacs.action = 'access_as_shared' AND
(networkrbacs.target_tenant = '*' OR networkrbacs.target_tenant = '9f13bf11e05e4d81b44c8c3e62d43628'))), action_network_stats AS (SELECT
DISTINCT nna.action, nna.network_id, COUNT(s.id) OVER (PARTITION BY nna.action, nna.network_id) AS subnet_count, STDDEV(s.ip_version)
OVER (PARTITION BY nna.action, nna.network_id) AS network_stddev_ip_version FROM non_shared_network_actions AS nna JOIN subnets AS s ON
s.network_id = nna.network_id WHERE s.enable_dhcp = 1), qualifying_actions AS (SELECT action FROM action_network_stats GROUP BY action
HAVING SUM(subnet_count) > 5), qualifying_network_stats AS (SELECT ans.* FROM action_network_stats AS ans JOIN qualifying_actions AS qa
ON qa.action = ans.action), action_rollup AS (SELECT CASE WHEN GROUPING(action) = 1 THEN 'GRAND TOTAL' ELSE 'ACTION SUBTOTAL' END AS
row_kind, CASE WHEN GROUPING(action) = 1 THEN NULL ELSE action END AS rbac_action, AVG(network_stddev_ip_version) AS
avg_stddev_ip_version, SUM(subnet_count)::bigint AS subnet_count, COUNT(*)::bigint AS network_count FROM qualifying_network_stats GROUP
BY ROLLUP(action)), active_shared_networks AS (SELECT n.id AS network_id, n.status AS network_status FROM networks AS n WHERE n.status =
'ACTIVE' AND EXISTS (SELECT 1 FROM networkrbacs AS nr WHERE nr.object_id = n.id AND nr.action = 'access_as_shared')),
network_subnet_counts AS (SELECT DISTINCT asn.network_id, asn.network_status, COUNT(s.id) OVER (PARTITION BY asn.network_id)::bigint AS
subnet_count FROM active_shared_networks AS asn LEFT JOIN subnets AS s ON s.network_id = asn.network_id), classified_networks AS (SELECT
nsc.network_id, nsc.network_status, nsc.subnet_count, CASE WHEN nsc.subnet_count > AVG(nsc.subnet_count) OVER () +
STDDEV(nsc.subnet_count) OVER () THEN 'High Subnet Count' ELSE 'Normal Subnet Count' END AS subnet_count_classification FROM
network_subnet_counts AS nsc), network_category_rollup AS (SELECT DISTINCT 'CATEGORY TOTAL'::text AS row_kind, network_status,
subnet_count_classification, COUNT(*) OVER (PARTITION BY network_status, subnet_count_classification)::bigint AS network_count FROM
classified_networks), network_status_rollup AS (SELECT DISTINCT 'STATUS SUBTOTAL'::text AS row_kind, network_status, NULL::text AS
subnet_count_classification, COUNT(*) OVER (PARTITION BY network_status)::bigint AS network_count FROM classified_networks),
network_grand_rollup AS (SELECT 'GRAND TOTAL'::text AS row_kind, NULL::text AS network_status, NULL::text AS
subnet_count_classification, COUNT(*)::bigint AS network_count FROM classified_networks), network_rollup AS (SELECT row_kind,
network_status, subnet_count_classification, network_count FROM network_category_rollup UNION ALL SELECT row_kind, network_status,
subnet_count_classification, network_count FROM network_status_rollup UNION ALL SELECT row_kind, network_status, NULL::text AS
subnet_count_classification, network_count FROM network_grand_rollup), combined AS (SELECT 'RBAC_ACTION_STDDEV'::text AS report_section,
ar.row_kind, ar.rbac_action, NULL::text AS network_id, NULL::text AS network_status, NULL::text AS subnet_count_classification,
ar.avg_stddev_ip_version, ar.subnet_count, ar.network_count FROM action_rollup AS ar UNION ALL SELECT 'ACTIVE_SHARED_NETWORKS',
'NETWORK', 'access_as_shared', cn.network_id, cn.network_status, cn.subnet_count_classification, NULL::double precision,
cn.subnet_count, 1::bigint FROM classified_networks AS cn UNION ALL SELECT 'ACTIVE_SHARED_NETWORKS', nr.row_kind, 'access_as_shared',
NULL::text, nr.network_status, nr.subnet_count_classification, NULL::double precision, NULL::bigint, nr.network_count FROM
network_rollup AS nr) SELECT report_section, row_kind, rbac_action, network_id, network_status, subnet_count_classification,
avg_stddev_ip_version, subnet_count, network_count FROM combined ORDER BY CASE report_section WHEN 'RBAC_ACTION_STDDEV' THEN 1 ELSE 2
END, CASE row_kind WHEN 'ACTION SUBTOTAL' THEN 1 WHEN 'NETWORK' THEN 1 WHEN 'CATEGORY TOTAL' THEN 2 WHEN 'STATUS SUBTOTAL' THEN 3 WHEN
'GRAND TOTAL' THEN 4 ELSE 5 END, rbac_action, network_status, subnet_count_classification, network_id
```

Figure 2: A candidate for BEAVER question neutron_152 [9] with 13 CTEs, 20 SELECTs; shown for its size, not to be read.

thirteen CTEs and combines window aggregates, a rollup, unions, and `NOT IN` and `EXISTS` subqueries. It runs and returns plausible rows, yet it does not answer the question. Few validators can assess such a query as one unit. Studies of SQL users find that errors concentrate on nested and correlated subqueries, self-joins, negation, and NULL semantics [1, 70, 75], and that users' ability to judge their own queries declines as queries grow [4]. LLMs falter on the same constructs [36, 50, 90]. Recent work supplies more evidence for this judgment, for example by translating the query back to natural language or by executing it on synthetic inputs [3], but the judgment still covers the whole query.

Our key insight is that a query is much easier to validate when it is simple for the validator, that is, when the validator can understand it easily. We therefore *decompose* the candidate query into smaller blocks tailored to the validator (shown as "blocks of DecoVal" in Figure 1). Each block is a SQL query that isolates one local validation task: the validator checks it against its own sub-question, using an interface that describes what each input computes and the constraints it carries. Together, the blocks recompose to exactly the candidate. Presenting a query as explained steps already raises the accuracy with which people validate generated SQL [74]; decomposition makes those steps precise and checkable.

Consider TPC-H Q17 [77], which asks for the average yearly revenue from lineitems whose quantity is below 20% of the average quantity for the same part, restricted to one brand and container. Judged as a whole, the candidate in Figure 3 requires tracking part eligibility, a correlated average, a per-part threshold, and a final aggregate simultaneously. The decomposition in the same figure asks four local questions instead: which parts are eligible; what is the threshold for a given part; which lineitems fall below their part's threshold; and what is their total revenue per year. Suppose a candidate omitted the correlation predicate `li.l_partkey = p.p_partkey` and thus compared every lineitem with 20% of the average quantity over *all* parts. Judged as a whole, the query still reads as a plausible threshold filter. Judged block by block, the defect stands out: the threshold block should return a value for a given part, yet its SQL takes no part as input.

Decomposition also introduces a fundamental tradeoff: smaller blocks reduce the load of each judgment, but every boundary adds a judgment, adds interface context to absorb, and risks separating information that a judgment needs. The best decomposition therefore depends on the validator. A user fluent in correlated aggregates prefers to see a threshold subquery together with its consumer; another finds it easier when the two are separate.

Challenges. Turning this insight into a solution raises four problems. (1) **Semantic preservation:** A decomposition must preserve the candidate's semantics exactly, including row multiplicities and any errors, so that judging its blocks amounts to judging the candidate. Relying on an LLM for decomposition can silently change what is being validated. (2) **Difficulty:** Choosing block boundaries requires a model of what makes a validation task hard for a particular validator, from the SQL inside a block and the inputs it reads to whether its output means something in terms of the question. There is also a trade-off between the difficulty of each task and the number of tasks. (3) **Search:** The space of decompositions grows with join orders, filter placements, and subquery boundaries. Moreover, the same task can occur at several places in a query but only needs to be judged once, which complicates local search decisions. (4) **Task assignment and composition:** Every block needs a requirement that reflects the user's intent, and a faithful description of what its inputs actually compute; the local verdicts must then combine into a verdict on the whole query.

Our Contributions. In this paper, we introduce DecoVal, a system that addresses these challenges with *validation-aware query decomposition*: just as a query optimizer chooses among equivalent plans by estimated execution cost, DecoVal chooses among semantics-preserving decompositions by how reliably a given validator is expected to judge them (Figure 4). We make the following contributions in this paper:

(1) **A formulation of validation-aware query decomposition:** We leverage the idea of decomposition to tackle the Reference-Free Validation problem, by formalizing validation-aware query decomposition as an optimization problem, with correctness guarantees and a clear reliability objective **(Section 3)**. This precise formulation allows the optimization space, objective, and search to be improved independently, beyond the solutions that this paper presents.

(2) **A model of the validation reliability:** We model how reliably a validator judges a candidate query through decomposed tasks from their SQL complexity, interface load, as well as coherence and relevance to the natural-language question. The overall optimization objective balances the reliability of each task against the number of judgments. We present a first instance of this model targeting human validators **(Section 5)**; alternatives can be readily plugged into our optimization framework.

(3) **Search space and strategy:** We show how a rich space of decompositions suitable for validation can be efficiently enumerated by applying sound rewrite rules to the candidate query's logical plan, and by picking appropriate "cut points" to partition it, such that every decomposition provably recomposes to the candidate query **(Section 4)**. We present an exact search algorithm that finds the optimum set of decomposed validation tasks by dynamic programming or, when tasks can be potentially shared, by A* **(Section 6)**.

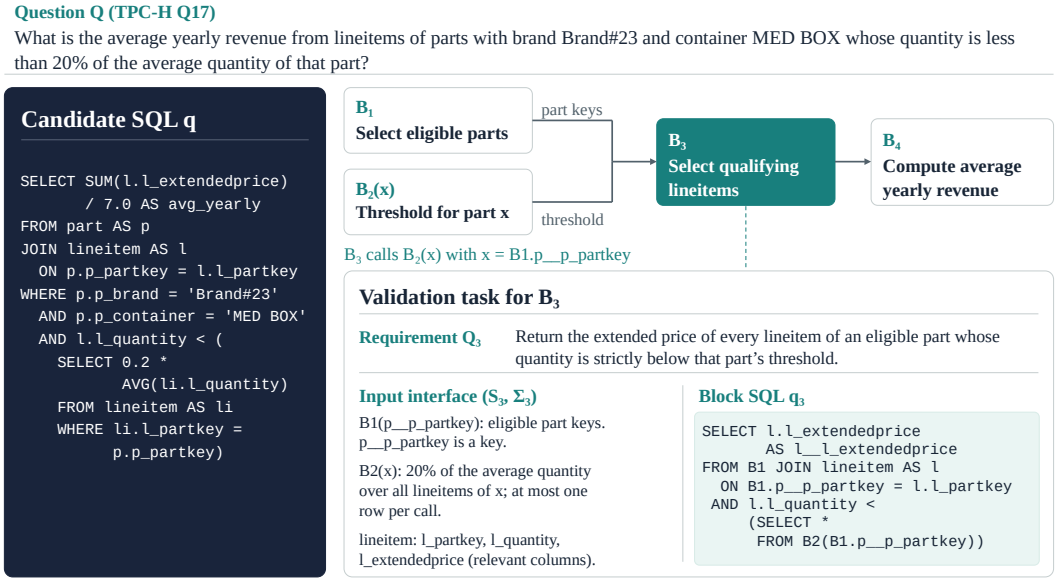


Figure 3: Running example. Left: a candidate SQL query. Right: a decomposition into four blocks and the validation task for B3. B2 is parameterized by a part key and returns that part's quantity threshold; B3 supplies the parameter when calling B2.

(4) **A user study with human validators:** We conducted a user study with 20 SQL-proficient students (volunteers among 218 we invited) in a supervised environment where only their validation skills (and no LLMs/Internet) were used. Each of them validated one LLM-generated candidate as a whole query and another decomposed by DecoVal. Decomposition increased the share of errors caught from 36% to 91%, with no false rejection, and they located the error in 9 of 10 rejections **(Section 8)**.

(5) **Experiments with LLM validators:** Although our primary motivation is to help human validators, it also helps LLM validators. On 305 questions from the BEAVER [9] and Spider 2.0 [34] benchmarks, DecoVal raises the error-detection recall of three LLMs by 7.0–16.9 percentage points compared with whole-query validation, while keeping precision above 90%. Therefore, any text-to-SQL method can benefit from DecoVal as an additional checking mechanism **(Section 9)**.

2 RELATED WORK

Semantic correctness of SQL. Alrashed et al. benchmark three algorithms for reference-free validation: SQL critique (direct LLM judgment), round-trip critique (back-translation into a question), and synthetic execution (LLM-generated test cases) [3]. Other methods detect semantic errors with database probes and LLM signals [19, 73] or with trained models [6, 10, 58], or abstain when a query is uncertain [7, 8]; NL2SQL-Bugs benchmarks such errors [41]. LLM judges [91] comparing two queries often miss a changed condition [59]. These techniques improve the evidence for one judgment and are orthogonal to DecoVal, which structures the judgment itself and can apply each of them to a block (Section 9.2).

Validation inside text-to-SQL generation. Text-to-SQL systems also validate implicitly, to choose or repair a query: they drop queries that fail or return no rows [80], vote over or cross-check execution results [12, 37, 82, 85], rank candidates with trained selectors or reward models [42, 49, 55, 88], or let the LLM correct itself [56, 66]. Such pipelines usually return their best candidate even when every candidate is wrong [3]. DecoVal judges a single candidate without alternatives or execution, so a pipeline can use it to accept or reject the query it returns.

Decomposition for SQL generation. DIN-SQL and DTS-SQL decompose the generation of SQL into stages such as schema linking [56, 57], QPL into the steps of a query plan [14], and CHASE-SQL into sub-questions [55]; these methods decompose to *produce* a query. Outside SQL, verifying the steps of mathematical reasoning is more reliable than verifying final answers or whole chains [39, 40]. We decompose a fixed candidate to *validate* it: the decomposition must preserve the candidate exactly, errors included, and its granularity is chosen for a validator rather than for a generator.

Helping people read and validate queries. Explaining a query in natural language lets users check that it matches their intent [33, 67], an idea that round-trip methods automate with LLMs [2, 3]. Interactive interfaces explain queries step by step, ask users about uncertain parts, or show sample data [13, 48, 72, 74, 86]; QueryVis and relational diagrams visualize nested queries [17, 35]; and reference-based tools explain wrong queries with counterexamples, abstract instances, repair hints, and provenance [18, 26, 27, 44]. Scrutinizer asks claim checkers about one query fragment at a time and plans its questions with a cost model of their effort [30]. Step-by-step explanations follow the candidate's clauses as written, so they cannot control what each step looks like. DecoVal chooses the blocks the validator prefers from equivalent forms of the candidate and gives each its own requirement, so the block verdicts decide the query.

Equivalence checking and testing. When a reference query exists, QED and SQLSolver prove equivalence for supported fragments [11, 81]; VeriEQL, SpotIt, and SpotIt+ search bounded instances, including under integrity constraints [24, 32, 78]; ParSEval and distilled test suites generate distinguishing databases [5, 92], as does test-data generation for complex queries [68]; and optimizer-based validation certifies some LLM rewrites and returns Unknown otherwise [47]. Equivalence is undecidable in general [76]. Benchmark references are themselves often wrong [29]. We need equivalence only between a candidate and its own recomposition, which holds by construction and is confirmed by checks on the printed SQL (Section 7); the comparison with intent is left to the validator, at a granularity it can handle.

Query rewriting and optimization. Rule-based rewriting [54], memo-based optimizers in the Volcano and Cascades tradition [20, 21], and join enumeration without cross products [45, 46] search equivalent plans for execution cost; equality saturation [71, 83, 84, 89] represents such spaces compactly. We reuse the memo but optimize a different objective: how reliably the blocks are judged, with a repeated task charged once. This resembles multi-query optimization, where a shared subexpression is computed once [62, 65] and optimal plan selection is NP-hard [64]. Our optimizer confines exponential search to the parts of a query where a task repeats.

Text-to-SQL benchmarks and models of difficulty. Workload studies measure query complexity by length and distinct operators [28, 79], and Spider and BIRD grade questions by hardness [38, 87]. Spider 2.0 grades its tasks by the length of the gold query [34], and BEAVER labels each query by its structure and by whether it requires domain knowledge [9]. None grades a validation task for a particular validator, as our item-response model does.

3 THE DECOVAL FRAMEWORK

The input of DecoVal consists of a natural-language question Q, a database schema S with constraints Σ (e.g., keys and foreign keys), and a candidate SQL query q over S. A *validator*, a person or an LLM, judges whether q correctly answers Q according to the validator's intended interpretation. Fix the validator's intended interpretation of Q. Let $\mathcal{G}(Q)$ be the set of queries over S that return the intended answer on every database instance satisfying Σ. A candidate q is *correct* for Q if $q \in \mathcal{G}(Q)$. A *validation task* $T = (Q, S, \Sigma, q)$ asks for a verdict—accept or reject—on whether q is correct for Q.

Note that the set $\mathcal{G}(Q)$ is never materialized: neither the system nor the validator has a reference query. Correctness validation is thus a semantic judgment, not a mechanical equivalence check.

3.1 Validation by Decomposition

Decomposition replaces one judgment of q against Q with local judgments, one for each block. In Figure 3, the validator of B3 checks that its SQL returns the lineitems of eligible parts whose quantity is below their part's threshold, taking the eligible parts from B1 and the thresholds from B2 as given. The validator thus reasons about one step of the query at a time. The correlated subquery, a construct on which people often err [1], becomes block B2, whose parameter and output are explicit.

Blocks and decompositions. A *block* is a SQL query over base tables and named relations produced by other blocks (its *inputs*), with optional *parameters* supplied by enclosing queries when the block is a correlated subquery. A *decomposition* D of q organizes blocks into a rooted directed acyclic graph (DAG), with edges from *producer blocks* to *reader blocks* that read their results as inputs. Each input has one producer, and the graph has a single sink, called the *root block*, that computes the final output. Compose(D) is a query obtained by substituting each producer's query for its references in dependency order, binding parameters at each call site.

Definition 3.1 (Semantics-preserving decomposition). We write $q \equiv_\Sigma q'$ when the two queries produce the same result on every instance of S satisfying Σ, including row multiplicities and, if specified, output ordering. A decomposition D of q is *valid* if Compose(D) $\equiv_\Sigma q$.

Block-wise validation and composite verdicts. Each block b in a valid decomposition D of q defines a separate *validation task*, $T_b = (Q_b, S_b, \Sigma_b, q_b)$. Here, q_b is the block's SQL, and Q_b is its natural-language requirement. The schema S_b describes its base tables, input relations, and parameters; Σ_b records the constraints that hold on S_b. DecoVal presents S_b and Σ_b to the validator as the context for judging q_b against Q_b. We call the tasks of D *consistent* if (1) for every block b' that reads a block b, the description of b's output in $S_{b'}$ matches Q_b; (2) for every block b, Σ_b follows from Σ and the requirements of the blocks that b reads, directly or indirectly; and (3) the requirement of the root block is Q.

A block b is *locally correct* if, on every database instance and parameter binding satisfying Σ_b, its SQL q_b satisfies its natural-language requirement Q_b *assuming that its inputs are given as described.* This assumption is important, as it allows judgment of T_b to proceed independently of validation of b's input blocks. The validator judges every task and returns accept or reject with a reason. The *composite verdict* accepts q iff every task is accepted; a rejection names the block and the requirement it violates.

Proposition 3.2 (Compositional soundness). *Let D be a valid decomposition of q for Q with consistent validation tasks. If every block of D is locally correct, then q is correct for Q.*

Proposition 3.2 follows directly from the definitions of validity, consistency, and local correctness. Each local judgment is self-contained: it needs the block's SQL, its requirement, and the descriptions of its inputs, and no verdict on another block. By contraposition of Proposition 3.2, if the tasks are consistent, every incorrect candidate has at least one block that is not locally correct, so a validator that judges every task correctly rejects every incorrect candidate and identifies a locally incorrect block.

3.2 The Decomposition Optimization Problem

A good decomposition should have four properties. It is *valid* (Definition 3.1), so that judging by parts enables judging the whole query. Its blocks are *simple*, each easy for the validator to understand, and *meaningful*, each computing a subgoal of the question that can be stated as a requirement. It involves *few judgments*: every block adds a task to judge, and a task that occurs at several places is judged once. While validity is a hard constraint, the other properties involve trade-offs. For example, fewer judgments may lead to more complex blocks, each of which can be harder to validate. Their balance depends on the validator, whose reliability depends on the

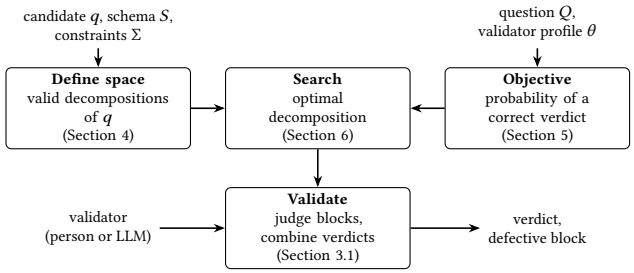


Figure 4: The pipeline of DecoVal. The search picks the decomposition that maximizes the validator's probability of a correct verdict, discounted by the number of judgments, and the validator then judges its blocks.

complexity of each task. Ultimately, we are interested in finding a decomposition that maximizes the reliability of the overall judgment. To model this trade-off and dependency on validators, we let θ denote a validator profile and $U_\theta(D)$ a modeled measure of validation reliability for decomposition D. We seek a valid decomposition that maximizes this measure among all candidates.

Definition 3.3 (Validation-aware Query Decomposition). Given a validation task $T = (Q, S, \Sigma, q)$, a validator profile θ, and a nonempty finite candidate set $\mathcal{D}(q)$ of valid decompositions of q, find

$$D^* \in \arg\max_{D \in \mathcal{D}(q)} U_\theta(D). \tag{1}$$

DecoVal solves this problem with the pipeline of Figure 4, which parallels the key components of traditional query optimization (but for a different objective): plan enumeration, cost estimation, and search procedure. For *plan enumeration*, we define the space of valid decompositions by applying sound rewrite rules to the candidate query's logical plan and finding a partitioning of plan operators into blocks. Instead of *cost estimation*, our objective U_θ measures how reliably the validator judges the tasks (Section 5). We score each task with an item-response model [60, 61], in which the chance of a correct judgment falls when the block's SQL complexity and interface load (the inputs and parameters it reads) exceed the validator's preferences and capabilities (as specified by θ), and if the block's natural-language requirement feels disconnected from Q. As validating the whole query requires judging all blocks in a decomposition, our model also penalizes the total number of judgments. As for the *search procedure*, we employ two methods (Section 6) for finding optimal decompositions: dynamic programming for decompositions in which no validation task can be reused, and A* for the general case (e.g., when the same filtered join can be cut out of two branches of a union).

Human vs. LLM validators. We want DecoVal to work for human validators, who we believe are ultimately responsible for interpreting the query intent. Hence, our modeling of U_θ in Section 5 is largely motivated by prior work on human reliability. Nonetheless, DecoVal is designed to see a validator only through its U_θ and verdicts, so LLM validators can be used just as easily. Our user study in Section 8 tests how DecoVal helps human users: whether blocks are easier to judge than whole queries, and how decomposition improves overall effectiveness of human validators. Section 9 further evaluates the effect of DecoVal with LLM validators, which are cheaper and allow larger-scale evaluation involving hundreds of candidate queries. As we will see in Section 9, DecoVal also helps in this setting, significantly improving other LLM-based validation methods.

4 THE DECOMPOSITION SPACE

The candidate set $\mathcal{D}(q)$ of Definition 3.3 determines which blocks the validator can receive. As a query optimizer considers plans beyond the written one, DecoVal expands the candidate's plan with sound rewrite rules into a data structure of equivalent subplans (Section 4.1) and forms blocks from these subplans at cut points (Section 4.2). Every decomposition obtained this way is valid, and the resulting space $\mathcal{D}_\mathcal{R}(q)$ serves as $\mathcal{D}(q)$.

4.1 Plans, Rewrites, and the Memo

In contrast to traditional query optimization for execution performance, where the space of execution plans can include those that do not resemble the original query at all, we take a more deliberate approach in enumerating decompositions, because it is beneficial for the validator to recognize the connection between the candidate query and the validation tasks. Hence, DecoVal considers only the plans that reorder the candidate's inner joins and move its filters within defined regions, preserving the main query structure. In particular, DecoVal never decorrelates a subquery; instead, it treats a block cut from a correlated subquery as parameterized, i.e., receiving values of its external column references as named input parameters (Section 4.2).

Skeleton plans. DecoVal imports the candidate query's logical plan into a *skeleton plan* defined using the following operators: *Scan*(T, o) reads an *occurrence* o of a base table T; *Join*(L, R, J) is an inner join with unordered inputs and a set J of *conjuncts* (more below); *Filter*(P, c) applies one conjunct c; other unary and binary operators, which we generally refer to as *peg* operators (which mark regions for rewriting, as described below), including: aggregation, generalized projections, `DISTINCT`, `ORDER BY`, `LIMIT`, windows, outer-, semi-, and anti-joins, set/bag operations, and references to CTEs (defined by `WITH`) and derived tables (subqueries in `FROM`). Note that an *occurrence* is one `FROM` item; multiple references to the same base table or CTE remain distinct occurrences. For example, in Q17, `l` and `li` are two occurrences of `lineitem`.

Normalization splits conjunctive query predicates into *conjuncts* (while keeping `OR` as atomic). We mark external column references in correlated subqueries as *free variables*, but otherwise allow them in expressions; i.e., no decorrelation is done. We also drop unnecessary projections that only remove or rename columns. We write P_0 for the normalized plan of q.

Region-based rewriting. Rewrites operate within each *region*, a maximal connected part of P_0 made of joins and filters within one `SELECT` statement. A region stops at its *inputs*, which are the scans and the subplans rooted at peg operators directly below it. The body of a subquery (which creates a new SQL scope) in a predicate or `SELECT` list also starts a region of its own. Rewrites do not move joins or filters moves out of their current region. For example, an aggregation (which is a peg operator) would separate the joins and filters below it from any filter above.

Within each region, DecoVal uses the join-reordering and filter-placement rules of query optimizers [54, 63]: a region's joins can be reordered in any way that creates no cross product, and each local conjunct can be applied at any point above its owner. The set $\mathcal{R}$ of rewrite rules is' ' as follows:

R1 $A \bowtie B \equiv B \bowtie A$ (join inputs are unordered);
R2 $(A \bowtie B) \bowtie C \equiv A \bowtie (B \bowtie C)$, applied only if the new inner join is spanned by some conjunct, so that no cross product is created;
R3 $Filter_{c_1}(Filter_{c_2}(A)) \equiv Filter_{c_2}(Filter_{c_1}(A))$;
R4 $Filter_c(A) \bowtie B \equiv Filter_c(A \bowtie B)$ if c (sans free variables) can be evaluated over A alone.

If every conjunct of q is deterministic and raises no runtime error, each rule preserves the output bag on every instance and every binding of the free variables.

Finally, within each region, DecoVal ensures that a join conjunct is pushed down as much as possible: i.e., it is applied at the *lowest* join whose subtree contains the inputs it needs (sans free variables). On the other hand, unlike traditional query optimization, DecoVal does not always push non-join filters down: intuitively, validating a join first and then a filter is not necessarily harder than validating a filtered table first and then a join involving it.

In general, rewrites expose subplans that the skeleton plan lacks without changing the shape of the candidate's SQL much; each exposed subplan can potentially become a block. By exploring different rewrites, DecoVal can, for example, break down a complex join involving many tables into simpler joins, or find opportunities for sharing the effort of validating common subplans.

The memo. DecoVal compactly represents the space of all rewritten plans in a *memo* [20] of *groups*: two subplans share a group exactly when R1–R4 rewrite one into the other. A group holds one or more subplans that accomplish the same computation but in different ways. Since DecoVal rules only reorder joins and move filters within a region, all subplans of a group read the same set of occurrences, apply the same conjuncts, and apply the same peg operators to the same child groups, but using different query expressions. For a group g, let $plans(g)$ denote the subplans obtained by choosing a query expression for g over its chosen recursively child subplans. For the root group g_0, which contains P_0, these are exactly the plans reachable from P_0 by R1–R4 across all regions. The memo is *sound*: all subplans in a group are logically equivalent.

4.2 Decomposition by Cut Points

Given a plan represented by the memo, we can further enumerate its valid decompositions (Section 3.1) by enumerating a set of *cut points* selected from the memo groups representing its subplans. Each selected cut point marks a block output, which becomes a named input to its readers; the plan fragments between these boundaries form the blocks. Hence, we can identify each block by a pair (g, I), where g is the memo group corresponding to the block's output, and I is the set of memo groups corresponding to its inputs.

Any subplan can be a cut point, subject to three rules. (1) Every block contains an operator other than scans, references, and column selection. The motivation for this rule is that every task should judge some computation; a bare scan or reference is too trivial to be a block by itself. (2) Within one `SELECT` statement, a cut may separate the `FROM/WHERE` part from the operators that follow it, as the boundary between B3 and B4 in Figure 3 does, but never falls among operators following `FROM/WHERE`—aggregation, `HAVING`, select list, `DISTINCT`, `ORDER BY`, and `LIMIT`. The motivation is that from our experience, cutting into post-`FROM/WHERE` steps is either unnecessary (e.g., substeps such as `DISTINCT` and `ORDER BY` are too small to warrant separate tasks) or too difficult (e.g., the result of grouping, separated from aggregation, cannot even be described as a flat table for verification). On the other hand, it is beneficial to consider cutting a complex `FROM/WHERE` part into simpler blocks. (3) The body of a CTE referenced multiple times in the candidate query is cut in every decomposition whenever it can be a block. Such a *forced cut* keeps the boundary that the candidate query already clearly draws; hence, the validator only needs to validate this CTE once as a separate subproblem.

Parameterized blocks. DecoVal makes it possible to create a block whose subplan contains free variables. This flexibility allows DecoVal to validate subqueries independently from the containing query. In this case, the free variables become explicit input parameters of the block; DecoVal will describe the meaning of these input parameters to the validator, enabling them to validate the block as a parameterized query instead of a standalone one. For example, in Figure 3, block B2 holds the threshold subquery with its free variable `p.p_partkey` replaced by a parameter x, so B2(x) returns the threshold of part x; B3 calls `B2(B1.p__p_partkey)` for each of its rows, binding x to that row's part key. In both contexts, B2 is readily interpreted as a function.

It is also worth noting that subqueries (correlated or not) do not automatically imply a new block or validation task. Indeed, more capable validators may tolerate or even prefer blocks containing some subqueries. The choice is up to DecoVal's decomposition objective (Section 5).

Search space. Let $\mathcal{D}_{\mathcal{R}}(q)$ be the set of decompositions that the rules $\mathcal{R}$ and the cut points yield. Each decomposition has one root block at g_0 and one producer per selected boundary, with all blocks reachable from the root.

Proposition 4.1 (Validity of the cut construction). *If every conjunct of q is deterministic and raises no runtime error, then for every decomposition $D \in \mathcal{D}_{\mathcal{R}}(q)$,* Compose($D$) *and P_0 return the same result on every instance, so D is valid in the sense of Definition 3.1.*

5 MODELING DECOMPOSITION OBJECTIVE

The objective U_θ plays the role of a query optimizer's cost model: it rates every decomposition in $\mathcal{D}_{\mathcal{R}}(q)$ by how reliably a validator is expected to judge it. Section 5.1 first describes how various features of a block can affect validator reliability. Section 5.2 then shows how to determine validator reliability for a given block using its features and the validator's profile. Section 5.3 discusses how to combine reliability across blocks into an overall optimization objective.

5.1 Block Features

A block can be hard to validate for various reasons; we break them down into three categories in Table 1, further discussed below.

SQL complexity. Every construct inside a block is a decision the validator must check against the intent: which rows a join pairs, which rows a predicate keeps, what a group contains, whether duplicates and `NULL`s are handled as intended, and which outer row

a correlated subquery refers to. The features count these constructs, one for each kind on which people and LLMs are known to err. Join omissions and wrong join conditions are among the most frequent logical errors [69, 70]; correlated subqueries and self-joins have the lowest success rates [1]; negation and NULL semantics mislead even database professionals [75]; and mixing Boolean connectives causes nesting errors [22, 70]. LLMs err most on set operations, subqueries, and joins [36, 50]. Finally, query length and operator diversity serve as proxies for the overall load [28, 53].

Interface load. A block b replaces part of the query with named input relations and parameters. The validator no longer sees how an input relation is computed or where in an outer block a parameter comes from; instead, the validator must now understand the meaning of such input relations and parameters in S_b, as part of the validation task $T_b = (Q_b, S_b, \Sigma_b, q_b)$. Furthermore, the validator must consider additional constraints in Σ_b associated with these input relations and parameters, which DecoVal propagates across block boundaries (Section 7). Therefore, we include the counts of input relations and parameters as measures of load placed on the validator to understand these interfaces.

Semantic score. Structural measures cannot tell whether a block computes a meaningful intermediate concept. For example, in Q17, applying the brand and container filters to `part` or to an input of already-joined `part-lineitem` pairs gives two blocks with the same two filtering predicates. The former directly produces the eligible-parts concept named by the question. A validator judges a block by mapping it to the intent, so it needs blocks whose outputs have an interpretable meaning (coherence) and are relevant to the overall question (relevance). Given a block b, DecoVal uses an LLM to obtain these two measures from its SQL q_b. Coherence, rated without the overall question, asks whether the relation makes sense on its own: 3 for a natural unit with a clear name and a clear row, such as the eligible parts; 2 if the name needs a qualification or some columns serve only a later step; 1 if the relation is describable only as an operation, such as the rows of one table joined with another, or its row is unclear; and 0 if it makes no sense alone, such as half of a condition. Relevance rates the relation against the overall question: *complete* if it computes a concept that the question names, with every condition the question attaches to it; *narrower* if some of these conditions are still to come; *technical* for a preparatory step that matches no phrase of the question; *split* if it applies a condition of the question only in part, such as one end of a range; and *unrelated* if it has no connection to the question. On each scale, the top level has factor 1, the second level, which still marks a sound unit, has factor $1 - \varepsilon$ with $\varepsilon = 0.05$, and the k-th level for $k \geq 3$ has factor 2^{2-k}. The semantic score $s(b) \in (0, 1]$ of a block b is the product of the coherence and relevance factors (the root block, whose natural-language requirement is the overall question, always has a semantic score of 1).

5.2 Validator Profile and Reliability Model

The validator profile θ captures the validator's preferences and/or capabilities: how much SQL of each kind, such as joins, predicates, or nested subqueries, it wants to judge within one block. Let X_θ denote the set of block features in Table 1 except coherence and relevance; θ assigns a threshold $\theta(x) \geq 0$ to every feature $x \in X_\theta$.

Table 1: Block features.

SQL complexity	
inner_join	inner joins ($m - 1$ for an m-input region)
outer_join	left, right, and full outer joins
pred	filter, join, `HAVING`, and `ON` conjuncts
bool_mix	`OR` and `NOT` operators inside conjuncts
negation	`NOT IN`, `NOT EXISTS`, `<> ALL`, negated predicates
null_sensitive	`IS [NOT] NULL`, `COALESCE`, `NULLIF`, `NOT IN`, outer-join columns
distinct	`DISTINCT`, `DISTINCT ON`, aggregates over `DISTINCT`
agg	grouping and global aggregation operators
expr	function calls, `CASE` branches, arithmetic, casts
setop	`UNION`, `INTERSECT`, `EXCEPT` operators
order_limit	`ORDER BY` and `LIMIT`/`OFFSET` operators
table_reuse	repeated uses of one relation (self-joins)
uncorr, corr	uncorrelated and correlated subquery calls
depth	subquery nesting depth inside the block
n_optypes	distinct operator types
length	tokens of the block
Interface load	
n_relations	distinct base tables and block inputs read
param	outer columns the block depends on
Semantic score (combined from below into $s(b) \in (0, 1]$)	
coherence	whether the block's output is meaningful on its own
relevance	whether the block's goal relates to the overall question

For example, one validator may be able to read a block joining four tables with ease, while another wants at most two-way joins; their profiles would assign different thresholds for inner_join. Section 1 gives a similar example for the correlated threshold of Q17. Generally speaking, raising a threshold admits larger and/or more complex blocks. The best profile will differ across validators, human and LLM alike: our human study (Section 8) reveals that some participants preferred smaller blocks while skilled ones preferred larger ones; our experiments with LLM validators (Section 9.4) similarly show that the best block granularity depends on the LLM.

Let $v_x(b)$ denote the value of feature $x \in X_\theta$ for block b, and let $s(b) \in (0, 1]$ denote its semantic score. Given a validator profile θ, the normalized excess of b in feature x is:

$$excess_x(\theta, b) = \frac{\max\{0,\, v_x(b) - \theta(x)\}}{\max\{\theta(x), 1\}}.$$

For constants $\beta > 0$ and $\gamma \geq 0$ of the model, the reliability of b is

$$F(\theta, b) = \underbrace{s(b)^\gamma}_{\text{Semantic factor}} \underbrace{\prod_{x \in X_\theta} \exp\big(-\beta\, excess_x(\theta, b)\big)}_{e^{-R(\theta,b)} \text{ for structural risk } R(\theta,b)}. \tag{2}$$

Here $R(\theta, b) = \beta \sum_{x \in X_\theta} excess_x(\theta, b)$ is the *structural risk* of b. For each feature $x \in X_\theta$, beyond the validator's threshold $\theta(x)$, an excess in feature value lowers validator reliability at a rate set by β; normalized excess lets one decay serve features of different scales. If every feature value is within the profile, $R(\theta, b) = 0$. Note that $R(\theta, b)$ is a conjunctive, non-compensatory multidimensional item-response model [61], where slack on one dimension does not offset an excess on another: e.g., misreading a single join or `NULL` test suffices to misjudge a block. Finally, the exponent γ weights a block's semantic score against its structural risk.

5.3 Optimization Objective

A block that is used multiple times as input in a decomposition only needs to be judged once. More generally, DecoVal detects

when two blocks can be associated with the same task: if they are equivalent up to a renaming of aliases, the verdict on one transfers to the other. We call the distinct tasks of a decomposition D its *task classes* $C(D)$. Since all blocks of a class c are essentially equivalent, we will with a slight abuse of notation use c in place of b in $F(\theta, c)$, $R(\theta, c)$, $s(c)$, etc.

The reliability of a decomposition multiplies the reliabilities of its distinct tasks and discounts the number of judgments it demands:

$$U_\theta(D) \;=\; e^{-\mu|C(D)|} \prod_{c \in C(D)} F(\theta, c), \tag{3}$$

where $\mu \geq 0$ prices one judgment. Maximizing this objective is equivalent to minimizing the total cost of the task classes:

$$J_\theta(D) \;=\; \sum_{c \in C(D)} w_\theta(c),$$
$$\text{where } w_\theta(c) = \mu - \log F(\theta, c) = \mu + R(\theta, c) - \gamma \log s(c). \tag{4}$$

The cost $w_\theta(c)$ adds the overhead of one judgment, the structural risk of its blocks, and their semantic cost.

Proposition 5.1 (Objective as a reliability bound). *If the validator's judgments of distinct tasks fail independently, and each task c is judged correctly with probability $F(\theta, c)$, then the composite verdict using consistent validation tasks for a valid decomposition is correct with probability at least $\prod_{c \in C(D)} F(\theta, c)$.*

Intuitively, this objective rewards decompositions that make each distinct task simpler and therefore more reliable to judge, it also prices the effort of each additional task. Consider, for example, a split of b into b_1, b_2 that removes b's task class and introduces two new, distinct classes, with all other class costs unchanged. The split pays off exactly when

$$\log F(\theta, b_1) + \log F(\theta, b_2) - \log F(\theta, b) > \mu.$$

One of the two blocks reads the other and keeps the output of b, and with it the semantic score of b. The split therefore pays off only if it lowers the structural risk by more than μ plus the semantic cost of the new block. Once every block is within the validator's thresholds, no split pays off, and among splits with equal structural gains, the objective prefers the one that cuts out the more meaningful block. A repeated task is paid once, so cutting it out of several places at once can pay off even when cutting it out of one place does not.

6 OPTIMAL DECOMPOSITION SEARCH

DecoVal finds a decomposition $D^* \in \mathcal{D}_\mathcal{R}(q)$ of minimum J_θ as a query optimizer finds a plan: it chooses a block for the root group g_0 and, recursively, for every input of a chosen block. If every task class held a single block, J_θ (Equation (4)) would add up the costs of the chosen blocks, and dynamic programming over memo groups would find the optimum (Section 6.1). A class may contain several blocks sharing the same validation task, however: e.g., when two branches of a union filter the same join. In this case, the cost of choosing the block on one branch then depends on whether the other does, similar to shared subexpressions in multi-query optimization [65]. Dynamic programming, which picks the best decomposition of each group on its own, can miss the optimum. We call such a class *shareable* and write C_{sh} for these classes; when $C_{sh} \neq \emptyset$, DecoVal uses A* search (Section 6.2).

6.1 DP for No Shareable Classes

When $C_{sh} = \emptyset$, every block of a decomposition is in its own class, so $J_\theta(D)$ is the sum of the costs of D's blocks. Recall from Section 4.2 that a block is represented by (g, I), where g is the memo group representing its output and I the set of memo groups representing its inputs. Given a target subquery g, to find $best(g)$, the minimum cost of decomposing g, we consider $\mathcal{B}$, the set of all candidate blocks in the valid decompositions of $\mathcal{D}_\mathcal{R}(q)$. The optimization boils down to picking the best input groups for the block outputting g:

$$best(g) = \min_{(g,I) \in \mathcal{B}} \Big[w_\theta(g, I) + \sum_{i \in I} best(i) \Big]. \tag{5}$$

Dynamic programming keeps one value $best(g)$ per memo group (along with the input groups I_{best} achieving this value), as in memoized plan enumeration [20, 63]. We call this procedure DP.

As a further optimization to the DP implementation, recall from Section 4.2 that we force-cut at groups corresponding to shared subqueries. Such groups can be resolved once and then excluded from DP enumeration and cost tallying in Equation (5); their total cost can be included in $best(g_0)$ at the end of DP.

Theorem 6.1 (Exactness without shareable classes). *If no class is shareable, DP returns a decomposition in $\mathcal{D}_\mathcal{R}(q)$ of minimum J_θ (proof in Appendix A.1).*

6.2 General A* Search

Algorithm 1 presents our A*-based algorithm [23] for the general case when some classes may be sharable. It builds a decomposition one block at a time, from the root down, and remembers which shareable classes it has already paid for. A search state $\sigma = (\Pi, \Phi)$ holds the pending groups Π, which still need to be further decomposed, and the shareable classes Φ paid so far. The search starts with the root group and the forced cuts pending and ends when no group is pending. A move takes the first pending group p, in a fixed topological order, and gives it a block (p, I) of some class c: p is then replaced by the input groups I, c is added to Φ if it is shareable, and the move costs $w_\theta(c)$, or nothing if $c \in \Phi$. States with the same (Π, Φ) have the same completions, so the search keeps only the cheapest path to each. A* expands states in order of $J(\sigma) + h(\sigma)$. Here $J(\sigma)$ is the part of J_θ paid so far, and $h(\sigma)$ estimates the rest: it adds up $best(p)$ over the pending groups p, computed as if every shareable class were free.

Besides the optimization involving forced cuts used also in DP (Section 6.1), Algorithm 1 applies two reductions in Fold to every new state to keep the search small. (1) A pending group below which no unpaid shareable class occurs is solved at once by DP with the classes of Φ free, since no choice inside it affects another group. (2) A paid class that no pending group can use is dropped from Φ, which merges states that differ only in past choices.

Theorem 6.2 (Exactness of the search). *The heuristic h is admissible and consistent, and Algorithm 1 returns a decomposition in $\mathcal{D}_\mathcal{R}(q)$ of minimum J_θ (proof in Appendix A.2).*

Proposition 6.3 (Cost). *Given $\mathcal{B}$, the search evaluates F once per class, at most $|\mathcal{B}|$ times. Without shareable classes, DP takes $O\big(\sum_{(g,I) \in \mathcal{B}} (1 + |I|)\big)$ time. With shareable classes, A* expands at*

Algorithm 1 Exact decomposition search

Require: blocks $\mathcal{B}$, class costs w
Ensure: a decomposition in $\mathcal{D}_R(q)$ of minimum J_θ

if no class is shareable **then return** the result of DP
$Q \leftarrow \{\text{Fold}(\text{start state})\}$
while $Q \neq \emptyset$ **do**
  take from Q the state $\sigma = (\Pi, \Phi)$ with least $J(\sigma) + h(\sigma)$
  if $\Pi = \emptyset$ **then return** the decomposition of σ
  $p \leftarrow$ the first pending group of σ
  for each block $(p, I) \in \mathcal{B}$, of class c **do**
    $\sigma' \leftarrow \sigma$ with p replaced by I (excluding forced cuts) and c paid
    add $\text{Fold}(\sigma')$ to Q
function $\text{Fold}(\sigma)$
  for each $p \in \Pi$ with no unpaid shareable class below it **do**
    solve p by DP with the classes of Φ free
  drop from Φ the classes no pending group can use; **return** σ

most $2^{|C_{sh}|}$ states for each set of pending groups that remain after folding.

The search is thus exponential only in the parts of a query where a task repeats. Enumerating $\mathcal{B}$ is the remaining cost: a block pairs the group it computes with a set of input cut points, so $|\mathcal{B}|$ can grow exponentially with the number of cut points of a region.

7 IMPLEMENTATION

DecoVal is implemented in Python; Figure 4 shows its pipeline. Semantic scoring for the objective and task assignment for validation call an LLM, the validator judges the resulting tasks, and the rest of the pipeline is deterministic. All stages before validation share a per-query time budget; a query that exceeds it receives no decomposition. Appendix B lists the prompts.

Import and memo. The importer parses a candidate with SQLGlot [43] in the PostgreSQL dialect into the skeleton plan of Section 4.1; it refuses syntactically invalid queries and queries with constructs that the skeleton plan does not represent, such as `LATERAL` joins, recursive CTEs, and table functions. A side table maps every conjunct and operator to its original SQL text. DecoVal builds the memo by equality saturation [71] in an egglog e-graph [84, 89]; once no rule adds a new plan, its e-classes are the groups of Section 4.1.

Task classes. Two blocks belong to the same task class if, after their aliases are renamed to `t0`, `t1`, … in a fixed order, they read the same tables and inputs and apply the same conjuncts and operators.

Scoring. For the structural risk, DecoVal counts the features of Table 1 directly on a class's plan fragment, without rendering its SQL. The LLM scorer sees each cut point as the SQL q_b of a block that outputs it, with the block's inputs expanded to base tables, and rates all of them in two calls, one per scale.

Rendering SQL for blocks. The printer builds each block's SQL from the side-table text, so the validator reads the candidate's own expressions. Proposition 4.1 holds for plans in the memo, and three checks confirm that the printed SQL preserves it. C1 re-imports the SQL printed for a memo group and requires the result to fall in the same group. C2 inlines the blocks of a decomposition into their readers, re-imports the result, and requires it to fall in the root group. C3 requires PostgreSQL to accept every printed query in a read-only `EXPLAIN` whenever it accepts the candidate. We say that DecoVal *supports* a query if it imports the query, returns a decomposition, and the printed SQL passes C1–C3.

Table 2: User study results. Time: mean/median. Located: the share of validations of an incorrect candidate that reject it at its defective block or at a block that reads its result.

	Whole		DecoVal		
Candidates	Accuracy	Time (s)	Accuracy	Located	Time (s)
Task 1					
correct	3/3	1418/1443	3/3	–	1344/1232
incorrect	**2/6**	649/520	**5/6**	**4/6**	950/975
Task 2					
correct	3/3	694/530	3/3	–	528/548
incorrect	**2/5**	339/258	**5/5**	**5/5**	834/564
Both tasks					
correct	6/6 (100%)	1056/947	6/6 (100%)	–	936/708
incorrect	**4/11** (36%)	508/486	**10/11** (91%)	**9/11**	897/931
all	**10/17** (59%)	701/530	**16/17** (94%)	–	911/823

Constraint propagation. DecoVal propagates declared schema constraints through memo groups and derives additional keys under bag semantics. Each operator has a rule that computes the constraints of its output from those of its inputs. For example, grouping by plain columns makes those columns a key. In Figure 3, `p__p_partkey` is a key of B1, because it is the primary key of `part` and a filter keeps the keys of its input. This key keeps B1's join with `lineitem` in B3 from duplicating a lineitem: the join condition equates it with `l.l_partkey`, so each lineitem matches at most one row of B1. B2 computes a global aggregate, so it returns at most one row per call, which makes its use as a scalar subquery in B3 valid. The task of B3 lists both facts in Σ_3. The rules are sound but not complete: every constraint they derive holds on every database that satisfies the declared constraints, so a task never states a false guarantee, but they can miss constraints that hold. Extending the rules to derive more constraints is future work.

Task assignment. For each non-root block of the selected decomposition, an LLM *writer* generates a requirement Q_b from Q, the relevant schema, the block's output columns and parameters, and the requirements and SQL of all its readers; the block's own SQL and derived keys are withheld. A separate LLM *describer* generates a description of each non-root block's output relation from a single SQL statement that composes the block with all its upstream producers; it describes the SQL as written. Blocks of one task class share requirement and description, so together with the two scoring calls, a decomposition with k task classes needs $2 + 2(k-1) = 2k$ LLM calls to give the validator k tasks, however many cut points the query has.

8 USER STUDY WITH HUMAN VALIDATORS

We test DecoVal with human validators and find validation on decomposition more effective for them than on the whole query: participants catch errors that they miss in whole queries, without rejecting correct candidates, and their rejections point to the errors.

Study Design. Our Campus Institutional Review Board (IRB) rules preclude a study within an ongoing database course or involving students advised by faculty, so we invited 218 SQL-proficient students who had taken an undergraduate or graduate database course

earlier (and offered a small gift card for their time); 20 of them volunteered. Participants took part in person, except a few who were not local and joined remotely, and were monitored over Zoom. None of them consulted an LLM. They validated candidate queries for two questions from BEAVER's dw split, Task 1 and Task 2, both marked as needing no domain knowledge. Each question had four candidate queries, two correct and two incorrect; we dropped one correct query per question because it received too few validations. Each participant performed a reference-free validation of one candidate of each task, Task 1 first, with the presentation counterbalanced: half of the participants validated Task 1 as a whole query and Task 2 decomposed, the other half the reverse. Candidates were assigned by balanced randomization, so every candidate was validated equally often both ways, giving 17 validations each way. A whole-query validation task shows the question, the schema, and the SQL. A decomposed task shows the blocks of DecoVal's decomposition, each with its requirement, the descriptions of its input blocks, the schema, and its SQL, and asks for each block whether its SQL does what its requirement asks; a candidate is rejected if any of its blocks is. We record each verdict, its explanation, and the time spent on each page. Appendix C gives more details on the questions.

Effect on Accuracy. Decomposition helps people catch errors. With DecoVal, participants rejected the incorrect candidates in 91% of the validations (10 of 11), against 36% (4 of 11) with whole queries (Table 2). On correct candidates, the two ways performed alike: participants accepted the candidate in all six validations with each. Overall, accuracy rose from 59% to 94%. This agrees with the LLM experiments (Section 9.2), with one difference: an LLM hallucinates defects with some probability and rejected two more correct candidates with DecoVal, whereas decomposition did not make human verdicts on correct candidates worse. The gain holds for both tasks. Task 1 is the harder validation task: with whole queries, only 2 of 6 validations of its incorrect candidates caught the error, and with DecoVal, 5 of 6. Task 2 is easier, yet whole-query validation still missed the error in 3 of 5 validations, while DecoVal caught all five.

Effect on Validation Time. With DecoVal, the time per block is stable: a block took a median of 233 s in Task 1 and 225 s in Task 2. The time of a whole query follows the task: a median of 726 s in Task 1, the harder task, in which participants also had to learn the schema that both tasks share, and 391 s in Task 2. On correct candidates, DecoVal took no longer than whole-query validation (708 vs. 947 s). On incorrect candidates, DecoVal took longer (931 vs. 486 s). A likely reason is that the participants who found the error in a whole query could make their judgment as soon as they saw the error and skipped the rest of the query, whereas with DecoVal, a participant who finds an error in an early block still validates the later ones. In practice, a validator can stop at the first rejected block, since one rejection rejects the candidate.

Localization of Errors. Decomposition also helps locate errors. Of the 10 rejections with DecoVal, 9 fell on the defective block or on a block that reads its result (Table 2), whereas a whole-query verdict carries no location. Participants were proficient in SQL: in 12 of their 14 rejections of incorrect candidates, the explanation named the defect, such as a `LIMIT 1` that keeps only one of several tied rows, a `CASE` branch that is never reached because an earlier condition covers it, or an integer division that truncates an average.

Table 3: The 305 candidate queries: mean (maximum).

	BEAVER	Spider 2.0	All
Questions	270	35	305
Correct / incorrect candidates	46 / 224	16 / 19	62 / 243
Tokens	210 (752)	206 (796)	209 (796)
Relations	3.2 (6)	3.0 (7)	3.2 (7)
Joins	2.7 (12)	3.0 (9)	2.8 (12)
Predicates	6.4 (18)	6.1 (26)	6.3 (26)
Aggregations	1.8 (9)	1.7 (6)	1.8 (9)
Subqueries	0.8 (7)	0.3 (3)	0.8 (7)
CTEs	2.6 (13)	2.7 (10)	2.6 (13)

9 EXPERIMENTS WITH LLM VALIDATORS

We evaluate whether decomposition helps validators judge candidate queries, and whether DecoVal chooses decompositions that suit them. LLM validators let us evaluate at scale by judging 305 candidate queries that a text-to-SQL system generates for questions from BEAVER [9] and Spider 2.0 [34], each under several decompositions and granularities, to answer three research questions:

(1) RQ1 (Comparison with Validation Baselines). Does DecoVal catch errors that baseline validation methods miss with the same LLM, and on which queries does it help most?

(2) RQ2 (Comparison of Decompositions). Do the decompositions that DecoVal chooses yield more accurate validation and simpler blocks than syntactic or random decompositions?

(3) RQ3 (Effect of Granularity across Validators). How does decomposition granularity affect different LLM validators?

9.1 Experimental Setup

All experiments ran on a Linux server with an AMD EPYC 7R13 processor (48 cores, 2.6 GHz) and 256 GB of RAM. Up to 8 candidates are processed in parallel, each with at most 4 concurrent LLM calls.

Benchmarks. Our benchmark has 305 questions. Of these, 270 come from the Neutron split of BEAVER [9], 1,017 queries over an enterprise database with 175 tables. BEAVER labels each query by its structure and by whether it requires domain knowledge; we draw 30 questions at random from each of the nine resulting categories, among those whose gold query DecoVal supports (Section 7). The other 35 are the Spider 2.0 [34] tasks on local databases whose gold query DecoVal supports. For each question, DIN-SQL [56] with GPT-6 Luna [52] at maximal reasoning effort generates one candidate query. The candidates are long and layered (Table 3): 209 tokens, 3.2 base tables, and 2.6 CTEs on average.

Labels. A candidate is *correct* if it passes its benchmark's official evaluation: for BEAVER, it returns the same set of rows as the gold query; for Spider 2.0, the benchmark's evaluator accepts its result. Of the 305 candidates, 62 are correct and 243 incorrect (BEAVER 46 and 224, Spider 2.0 16 and 19).

Validators and configuration. Three LLMs serve as validators: GPT-5.5, GPT-5.6 Sol, and GPT-6 Astra [52], each at medium reasoning effort. Every LLM call runs through the Codex CLI [51] in a sandbox that holds only its prompt, so a validator sees only its task, never the gold query, the labels, or a previous result. The reliability model uses $\beta = \ln 2$, $\gamma = 0.25$, and $\mu = 10^{-3}$, with default thresholds τ^*: 2 for inner_join, bool_mix, and order_limit, 3 for pred, 4 for expr and n_relations, 5 for n_optypes, 150 tokens for length,

and 1 for every other feature (Table 1). We set τ^* before the user study from the constructs on which people and LLMs most often err (Section 5.1) and our own judgment of SQL difficulty. GPT-6 Luna writes requirements and input descriptions at maximal reasoning effort and rates coherence and relevance at high effort. The time budget of Section 7 is one hour per candidate. To vary granularity, we scale every threshold by a factor α, $\tau = \alpha\tau^*$: a larger α models a validator with a wider range and yields coarser decompositions. Unless stated otherwise, the validator is GPT-5.6 Sol and $\alpha = 1$.

Validation baselines. **1. Whole** validates the candidate as a whole against the question, without decomposing it. Whole $\times k$ runs Whole k times, where k is the number of distinct blocks in DecoVal's decomposition of the candidate, and rejects the candidate if any run rejects it. We also run two validation methods of Alrashed et al. [3]. **2. *Round-trip critique* (RT)** has the LLM translate the candidate back into a question without seeing the original question, and accepts the candidate if the LLM gives the two questions a similarity score of at least 4 out of 5. **3. *Synthetic execution* (SE)** has the LLM write the expected answer to the question and then a database instance that yields it, executes the candidate on that instance, and accepts the candidate if its result equals the expected answer. DecoVal + RT and DecoVal + SE apply RT and SE to each of DecoVal's blocks instead of the whole candidate, with the block's requirement as the question and its inputs as tables; a block that SE cannot execute keeps DecoVal's verdict.

Decomposition strategies. We compare DecoVal with three other decomposition strategies: Syntactic cuts at every CTE, derived table, and subquery written in the candidate where the cut is legal; Random-k randomly samples a legal decomposition with as many distinct blocks as DecoVal's; and DecoVal with $\gamma = 0$ optimizes the structural risk model alone. All decompositions keep the forced cuts of Section 4.2 and share task assignment, presentation, and verdict composition, so they differ only in their blocks.

Metrics. We take an incorrect candidate as the positive class, so a true positive is an error that the validator detects (Appendix D):

	Validator rejects	Validator accepts
Incorrect candidate	TP	FN
Correct candidate	FP	TN

We report accuracy, precision, recall (the share of incorrect candidates rejected), specificity (the share of correct candidates accepted), and F1. No method judges the five candidates that DecoVal does not support (four use `LATERAL`, one a table function). A candidate without a verdict stays in the denominators but in no numerator.

Token cost. We report medians per candidate, with GPT-5.6 Sol as the validator. DecoVal makes 9 LLM calls: two for semantic scores, four to write requirements and input descriptions, and three validator calls. At list prices [52], validating a query costs \$0.08 using DecoVal, against \$0.02 for Whole. Of DecoVal's cost, \$0.01 goes to the semantic scores, requirements, and input descriptions, all computed once per question–query pair and shared by every validator. Runtimes are not comparable across methods because Codex's rate limits throttled our concurrent calls.

9.2 RQ1: Comparison with Validation Baselines

Decomposition vs. whole queries. With the same LLM validator, DecoVal catches many errors that validating the whole query misses

Table 4: Validation outcomes on 305 candidates.

	Candidates				Measures (%)				
Method	TP	FN	FP	TN	Acc	Prec	Rec	Spec	F1
Without execution									
Whole	61	182	6	55	38.0	91.0	25.1	**88.7**	39.4
Whole $\times k$	83	160	6	55	45.2	**93.3**	34.2	**88.7**	50.0
DecoVal	97	146	8	53	**49.2**	92.4	**39.9**	85.5	**55.7**
RT	43	200	3	58	33.1	**93.5**	17.7	**93.5**	29.8
DecoVal + RT	153	90	26	35	**61.6**	85.5	**63.0**	56.5	**72.5**
With execution									
SE	188	55	33	27	70.5	**85.1**	77.4	**43.5**	81.0
DecoVal + SE	220	23	46	15	**77.0**	82.7	**90.5**	24.2	**86.4**

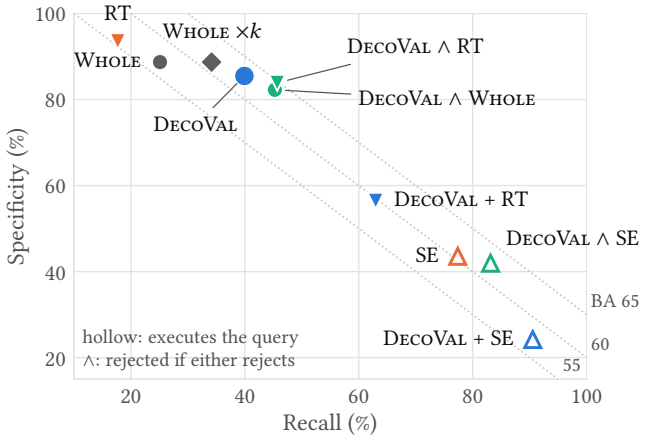


Figure 5: Recall and specificity of the methods of Table 4 and their combinations with DecoVal. Dotted lines mark equal balanced accuracy (BA), the mean of recall and specificity.

(Table 4, Figure 5): it rejects 97 of the 243 incorrect candidates against 61 for Whole, raising recall from 25.1% to 39.9% and F1 from 39.4 to 55.7 with two more false rejections. Decomposition contributes detections beyond those of repeated judgments: Whole $\times k$, which makes as many judgments as DecoVal and composes them with the same rule, reaches a recall of 34.2% and an F1 of 50.0, yet still finds 14 fewer errors than DecoVal.

Decomposition also raises the recall of the two methods of Alrashed et al., which err in opposite directions on whole queries. RT rarely rejects because comparing two natural-language questions misses many differences that change the result: it catches 43 incorrect candidates and gives 89 of the 200 it misses a similarity score of 5. SE catches 188 incorrect candidates but rejects 33 of the 62 correct ones. On DecoVal's blocks, RT catches 153 incorrect candidates and SE 220, or 90.5% of all incorrect queries, while their false rejections rise from 3 to 26 and from 33 to 46. Decomposition thus raises the BA of the LLM validator and RT, from 56.9 to 62.7 and from 55.6 to 59.7, and lowers that of SE, from 60.5 to 57.4.

Where decomposition helps most. Decomposition helps most on complex queries, as the reliability model predicts. Figure 6 groups the candidates by the structural risk R of the whole query (Section 5.2), which grows as the query's features exceed the validator's thresholds. The recall of DecoVal exceeds that of Whole by 12.5

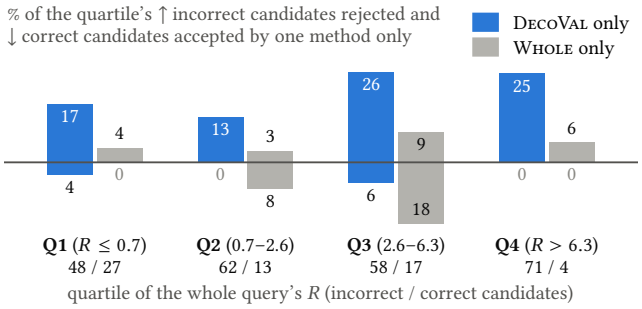


Figure 6: Where DecoVal and Whole disagree, by quartile of the whole query's structural risk R.

Table 5: Decomposition strategies. # blocks: mean distinct blocks per candidate; Out of range: blocks with a feature over its threshold in τ^*; R: mean structural risk of a block.

	Decomposition			Measures (%)		
Strategy	# blocks	Out of range (%)	R	Rec	Spec	BA
Whole	1.0	92.0	4.34	25.1	88.7	56.9
Syntactic	4.4	22.9	0.27	42.0	74.2	58.1
Random-k	4.0	30.6	0.34	39.1	83.9	61.5
DecoVal, $\gamma = 0$	3.9	**13.1**	**0.13**	**42.4**	80.6	61.5
DecoVal	4.0	13.4	**0.13**	39.9	**85.5**	**62.7**

and 9.7 points in the lower two quartiles and by 17.2 and 19.7 points in the upper two, where DecoVal alone rejects 33 incorrect candidates and Whole alone 9. In every quartile, DecoVal has at most two more false rejections than Whole. Blocks and whole queries complement each other. Whole catches 13 errors that DecoVal misses, some of which concern how the steps fit together and are easier to see on the whole query. In one BEAVER candidate, a left join keeps unmatched rows; the aggregates skip their NULL values, but a later grouping turns each of them into an extra row of the result. DecoVal puts the join and the grouping in separate blocks with correct requirements, and each looks right on its own; Whole sees both together and rejects the candidate. Combining verdicts by conjunction (∧) catches more errors: DecoVal ∧ Whole rejects 110 incorrect candidates and DecoVal ∧ RT 111, against 97 for DecoVal, with one or two more false rejections; DecoVal ∧ RT reaches the highest BA (64.8) in Figure 5.

9.3 RQ2: Comparison of Decompositions

DecoVal produces the fewest and simplest blocks and the best validation (Table 5). It cuts a candidate into 3.9 to 4.0 blocks on average, against 4.4 for Syntactic, and under the reliability model it optimizes, its blocks carry less than half the structural risk of those of Syntactic and Random-k. With $\gamma = 0$ it catches the most errors, and by default it reaches the highest BA, 62.7. Syntactic depends on how the query is written. DIN-SQL writes 238 of the 305 candidates as pipelines of CTEs, which give Syntactic natural places to cut; on the other 67 it finds few, and its blocks carry six times the structural risk of DecoVal's.

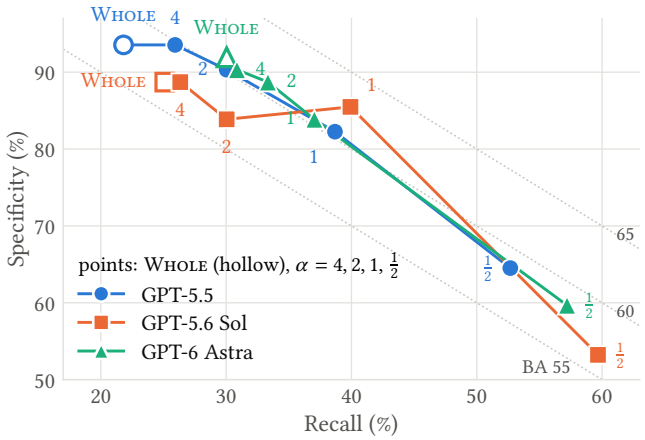


Figure 7: Recall and specificity of three LLM validators from the whole query to $\alpha = \frac{1}{2}$. Dotted lines join equal BA.

Random-k leaves the difficulty of its blocks to chance: with as many blocks as DecoVal, its riskiest block carries more than five times the structural risk of DecoVal's on the tenth of the candidates where the two differ most. The structural risk alone accounts for DecoVal's advantage: without the semantic score ($\gamma = 0$), its blocks are as simple, and the semantic score trades 6 detections for 3 fewer false rejections.

9.4 RQ3: Effect of Granularity across Validators

Granularity trades recall against specificity for every validator (Figure 7). DecoVal rejects a candidate as soon as it rejects one block, so every additional judgment is another chance to reject. From Whole to $\alpha = \frac{1}{2}$, where a candidate has 7.9 blocks on average, each validator gains 27 to 35 points of recall and loses 29 to 35 points of specificity. Conversely, a larger α makes fewer judgments and raises specificity toward that of Whole.

Recall and specificity move along a line of nearly equal BA for GPT-5.5 and GPT-6 Astra, whose BA varies by less than 3 points across granularities; GPT-5.6 Sol has a clear optimum at $\alpha = 1$, 5.8 points above Whole and at least 5.2 above any other granularity. The choice of granularity thus depends on the validator.

10 CONCLUSION AND FUTURE WORK

Validation-aware query decomposition rewrites a candidate query into simpler blocks that recompose to it by construction and chooses the decomposition that a given validator is expected to judge most reliably. It more than doubled the share of errors that participants in our user study caught, from 36% to 91%, and raised the share that three LLM validators caught by 7.0–16.9 points. Future work can improve three parts of DecoVal. Its rewrites only reorder joins and move filters, which keeps blocks close to the candidate's SQL; whether decorrelation and other rewrites help validators is open. Its reliability model uses default thresholds and could be fitted to a validator's past verdicts. Its block requirements and descriptions come from LLMs; finding more reliable ways to assign tasks to blocks is also an open problem.

REFERENCES

[1] Alireza Ahadi, Julia Prior, Vahid Behbood, and Raymond Lister. 2015. A Quantitative Study of the Relative Difficulty for Novices of Writing Seven Different Types of SQL Queries. In *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education* (Vilnius, Lithuania) *(ITiCSE '15)*. Association for Computing Machinery, New York, NY, USA, 201–206. https://doi.org/10.1145/2729094.2742620

[2] Miltiadis Allamanis, Sheena Panthaplackel, and Pengcheng Yin. 2024. Unsupervised evaluation of code LLMs with round-trip correctness. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 44, 17 pages.

[3] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. 2026. Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation. *Proc. VLDB Endow.* 19, 11 (Sept. 2026), 3732–3744. https://doi.org/10.14778/3836663.3836721

[4] Gretchen I. Casterella and Leo Vijayasarathy. 2013. An Experimental Investigation of Complexity in Database Query Formulation Tasks. *Journal of Information Systems Education* 24, 3 (2013), 211–222. https://jise.org/volume24/n3/JISEv24n3p211.html

[5] Chunyu Chen, Zhengjie Miao, Yong Zhang, and Jiannan Wang. 2025. ParSEval: Plan-Aware Test Database Generation for SQL Equivalence Evaluation. *Proc. VLDB Endow.* 18, 11 (July 2025), 4750–4762. https://doi.org/10.14778/3749646.3749727

[6] Jikai Chen, Leilei Gan, Ziyu Zhao, Zechuan Wang, Dong Wang, and Chenyi Zhuang. 2025. SQLCritic: Correcting Text-to-SQL Generation via Clause-wise Critic. arXiv:2503.07996 [cs.AI] https://arxiv.org/abs/2503.07996

[7] Kaiwen Chen, Yueting Chen, Nick Koudas, and Xiaohui Yu. 2025. Reliable Text-to-SQL with Adaptive Abstention. *Proc. ACM Manag. Data* 3, 1, Article 69 (Feb. 2025), 30 pages. https://doi.org/10.1145/3709719

[8] Kaiwen Chen, Yueting Chen, Nick Koudas, and Xiaohui Yu. 2026. Reliable End-to-End Text-to-SQL Generation. In *Proceedings 29th International Conference on Extending Database Technology (EDBT 2026)* (Tampere, Finland) *(Advances in Database Technology)*, Vol. 29. OpenProceedings.org, 544–550. https://doi.org/10.48786/edbt.2026.44

[9] Peter Baile Chen, Devin Yang, Weiyue Li, Fabian Wenz, Yi Zhang, Nesime Tatbul, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2026. BEAVER: An Enterprise Benchmark for Text-to-SQL. arXiv:2409.02038 [cs.CL] https://arxiv.org/abs/2409.02038

[10] Shijie Chen, Ziru Chen, Huan Sun, and Yu Su. 2023. Error Detection for Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 11730–11743. https://doi.org/10.18653/v1/2023.findings-emnlp.785

[11] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. 2023. Proving Query Equivalence Using Linear Integer Arithmetic. *Proc. ACM Manag. Data* 1, 4, Article 227 (Dec. 2023), 26 pages. https://doi.org/10.1145/3626768

[12] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306 [cs.CL] https://arxiv.org/abs/2307.07306

[13] Ahmed Elgohary, Saghar Hosseini, and Ahmed Hassan Awadallah. 2020. Speak to your Parser: Interactive Text-to-SQL with Natural Language Feedback. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 2065–2077. https://doi.org/10.18653/v1/2020.acl-main.187

[14] Ben Eyal, Moran Mahabi, Ophir Haroche, Amir Bachar, and Michael Elhadad. 2023. Semantic Decomposition of Question and SQL for Text-to-SQL Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 13629–13645. https://doi.org/10.18653/v1/2023.findings-emnlp.910

[15] Avrilia Floratou, Fotis Psallidas, Fuheng Zhao, Shaleen Deep, Gunther Hagleither, Wangda Tan, Joyce Cahoon, Rana Alotaibi, Jordan Henkel, Abhik Singla, Alex van Grootel, Brandon Chow, Kai Deng, Katherine Lin, Marcos Campos, Venkatesh Emani, Vivek Pandit, Victor Shnayder, Wenjing Wang, and Carlo Curino. 2024. NL2SQL is a solved problem... Not!. In *14th Annual Conference on Innovative Data Systems Research (CIDR '24)* (Chaminade, California, USA). www.cidrdb.org. https://www.cidrdb.org/cidr2024/papers/p74-floratou.pdf

[16] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[17] Wolfgang Gatterbauer and Cody Dunne. 2024. On The Reasonable Effectiveness of Relational Diagrams: Explaining Relational Query Patterns and the Pattern Expressiveness of Relational Languages. *Proc. ACM Manag. Data* 2, 1, Article 61 (March 2024), 27 pages. https://doi.org/10.1145/3639316

[18] Amir Gilad, Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2022. Understanding Queries by Conditional Instances. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 355–368. https://doi.org/10.1145/3514221.3517898

[19] Yue Gong, Chuan Lei, Xiao Qin, Kapil Vaidya, Balakrishnan Murali Narayanaswamy, and Tim Kraska. 2025. SQLens: An End-to-End Framework for Error Detection and Correction in Text-to-SQL. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=on6Hf0KP20

[20] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Eng. Bull.* 18, 3 (1995), 19–29. http://sites.computer.org/debull/95SEP-CD.pdf

[21] Goetz Graefe and William J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *Proceedings of the Ninth International Conference on Data Engineering*. IEEE Computer Society, USA, 209–218.

[22] S. L. Greene, S. J. Devlin, P. E. Cannata, and L. M. Gomez. 1990. No IFs, ANDs, or ORs: A study of database querying. *Int. J. Man-Mach. Stud.* 32, 3 (March 1990), 303–326. https://doi.org/10.1016/S0020-7373(08)80005-3

[23] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107. https://doi.org/10.1109/TSSC.1968.300136

[24] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. 2024. VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 132 (April 2024), 29 pages. https://doi.org/10.1145/3649849

[25] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering* 37, 12 (2025), 7328–7345. https://doi.org/10.1109/TKDE.2025.3609486

[26] Yihao Hu, Zian Chen, Zhiming Leong, Sharan Sokhi, Zachary Zheng, Alex Chao, Kristin Stephens-Martinez, Sudeepa Roy, and Jun Yang. 2026. I-Rex: An Interactive Debugger for SQL. *Proc. VLDB Endow.* 19, 11 (Sept. 2026), 3020–3033. https://doi.org/10.14778/3836663.3836670

[27] Yihao Hu, Amir Gilad, Kristin Stephens-Martinez, Sudeepa Roy, and Jun Yang. 2024. Qr-Hint: Actionable Hints Towards Correcting Wrong SQL Queries. *Proc. ACM Manag. Data* 2, 3, Article 164 (May 2024), 27 pages. https://doi.org/10.1145/3654995

[28] Shrainik Jain, Dominik Moritz, Daniel Halperin, Bill Howe, and Ed Lazowska. 2016. SQLShare: Results from a Multi-Year SQL-as-a-Service Experiment. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 281–293. https://doi.org/10.1145/2882903.2882957

[29] Tengjun Jin, Yoojin Choi, Yuxuan Zhu, and Daniel Kang. 2026. Pervasive Annotation Errors Break Text-to-SQL Benchmarks and Leaderboards. *Proc. VLDB Endow.* 19, 5 (May 2026), 931–944. https://doi.org/10.14778/3796195.3796206

[30] Georgios Karagiannis, Mohammed Saeed, Paolo Papotti, and Immanuel Trummer. 2020. Scrutinizer: a mixed-initiative approach to large-scale, data-driven claim verification. *Proc. VLDB Endow.* 13, 12 (July 2020), 2508–2521. https://doi.org/10.14778/3407790.3407841

[31] George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A survey on deep learning approaches for text-to-SQL. *The VLDB Journal* 32, 4 (Jan. 2023), 905–936. https://doi.org/10.1007/s00778-022-00776-8

[32] Rocky Klopfenstein, Yang He, Andrew Tremante, Yuepeng Wang, Nina Narodytska, and Haoze Wu. 2026. SpotIt: Evaluating Text-to-SQL Evaluation with Formal Verification. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=iMkvR2ICSE

[33] Georgia Koutrika, Alkis Simitsis, and Yannis E. Ioannidis. 2010. Explaining structured queries in natural language. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. 333–344. https://doi.org/10.1109/ICDE.2010.5447824

[34] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=XmProj9cPs

[35] Aristotelis Leventidis, Jiahui Zhang, Cody Dunne, Wolfgang Gatterbauer, H.V. Jagadish, and Mirek Riedewald. 2020. QueryVis: Logic-based Diagrams help Users Understand Complicated SQL Queries Faster. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 2303–2318. https://doi.org/10.1145/3318464.3389767

[36] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Proc. VLDB Endow.* 17, 11 (July 2024), 3318–3331. https://doi.org/10.14778/3681954.3682003

[37] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: zero-shot text-to-SQL using Monte Carlo tree search. In

Proceedings of the 42nd International Conference on Machine Learning (Vancouver, Canada) *(ICML'25)*. JMLR.org, Article 1456, 21 pages.

[38] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1835, 28 pages.

[39] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=v8L0pN6EOi

[40] Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. 2023. Deductive verification of chain-of-thought reasoning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1580, 27 pages.

[41] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. 2025. NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2* (Toronto ON, Canada) *(KDD '25)*. Association for Computing Machinery, New York, NY, USA, 5662–5673. https://doi.org/10.1145/3711896.3737427

[42] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, and Jingren Zhou. 2026. XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering* 38, 4 (2026), 2474–2487. https://doi.org/10.1109/TKDE.2026.3657851

[43] Toby Mao. 2023. SQLGlot: A no-dependency SQL parser, transpiler, optimizer, and engine. https://github.com/tobymao/sqlglot

[44] Zhengjie Miao, Sudeepa Roy, and Jun Yang. 2019. Explaining Wrong Queries Using Small Examples. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 503–520. https://doi.org/10.1145/3299869.3319866

[45] Guido Moerkotte and Thomas Neumann. 2006. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products. In *Proceedings of the 32nd International Conference on Very Large Data Bases* (Seoul, Korea) *(VLDB '06)*. VLDB Endowment, 930–941.

[46] Guido Moerkotte and Thomas Neumann. 2008. Dynamic programming strikes back. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data* (Vancouver, Canada) *(SIGMOD '08)*. Association for Computing Machinery, New York, NY, USA, 539–552. https://doi.org/10.1145/1376616.1376672

[47] Vivek Narasayya and Surajit Chaudhuri. 2026. Leveraging Query Optimizers to Verify the Soundness of LLM-based Query Rewrites for Real-World Workloads, and More!. In *16th Annual Conference on Innovative Data Systems Research (CIDR '26)* (Chaminade, California, USA). www.cidrdb.org. https://www.cidrdb.org/cidr2026/papers/p33-narasayya.pdf

[48] Arpit Narechania, Adam Fourney, Bongshin Lee, and Gonzalo Ramos. 2021. DIY: Assessing the Correctness of Natural Language to SQL Systems. In *Proceedings of the 26th International Conference on Intelligent User Interfaces* (College Station, TX, USA) *(IUI '21)*. Association for Computing Machinery, New York, NY, USA, 597–607. https://doi.org/10.1145/3397481.3450667

[49] Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: learning to verify language-to-code generation with execution. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 1086, 23 pages.

[50] Zheng Ning, Yuan Tian, Zheng Zhang, Tianyi Zhang, and Toby Jia-Jun Li. 2024. Insights into Natural Language Database Query Errors: from Attention Misalignment to User Handling Strategies. *ACM Trans. Interact. Intell. Syst.* 14, 4, Article 25 (Dec. 2024), 32 pages. https://doi.org/10.1145/3650114

[51] OpenAI. 2025. Codex CLI. https://github.com/openai/codex Accessed October 2026.

[52] OpenAI. 2026. OpenAI API Models. https://developers.openai.com/api/docs/models Accessed October 2026.

[53] Norman Peitek, Sven Apel, Chris Parnin, André Brechmann, and Janet Siegmund. 2021. Program Comprehension and Code Complexity Metrics: An fMRI Study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 524–536. https://doi.org/10.1109/ICSE43902.2021.00056

[54] Hamid Pirahesh, Joseph M. Hellerstein, and Waqar Hasan. 1992. Extensible/rule based query rewrite optimization in Starburst. In *Proceedings of the 1992 ACM SIGMOD International Conference on Management of Data* (San Diego, California, USA) *(SIGMOD '92)*. Association for Computing Machinery, New York, NY, USA, 39–48. https://doi.org/10.1145/130283.130294

[55] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. 2025. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=CvGqMD5OtX

[56] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1577, 10 pages.

[57] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8212–8220. https://doi.org/10.18653/v1/2024.findings-emnlp.481

[58] Rihong Qiu, Zhibang Yang, Xinke Jiang, Weibin Liao, Xin Gao, Xu Chu, Junfeng Zhao, and Yasha Wang. 2025. Bridging Global Intent with Local Details: A Hierarchical Representation Approach for Semantic Validation in Text-to-SQL. arXiv:2512.22744 [cs.LG] https://arxiv.org/abs/2512.22744

[59] Ananya Rahman, Anny Zheng, Mostafa Milani, Fei Chiang, and Rachel Pottinger. 2025. Evaluating SQL Understanding in Large Language Models. https://doi.org/10.48786/EDBT.2025.74

[60] G. Rasch. 1960. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danmarks Paedagogiske Institut. https://books.google.com/books?id=XnlevwEACAAJ

[61] Mark D. Reckase. 2009. *Multidimensional Item Response Theory*. Springer, New York, NY. https://doi.org/10.1007/978-0-387-89976-3

[62] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhobe. 2000. Efficient and extensible algorithms for multi query optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (Dallas, Texas, USA) *(SIGMOD '00)*. Association for Computing Machinery, New York, NY, USA, 249–260. https://doi.org/10.1145/342009.335419

[63] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (Boston, Massachusetts) *(SIGMOD '79)*. Association for Computing Machinery, New York, NY, USA, 23–34. https://doi.org/10.1145/582095.582099

[64] T. Sellis and S. Ghosh. 1990. On the Multiple-Query Optimization Problem. *IEEE Trans. on Knowl. and Data Eng.* 2, 2 (June 1990), 262–266. https://doi.org/10.1109/69.54724

[65] Timos K. Sellis. 1988. Multiple-query optimization. *ACM Trans. Database Syst.* 13, 1 (March 1988), 23–52. https://doi.org/10.1145/42201.42203

[66] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 377, 19 pages.

[67] Alkis Simitsis and Yannis Ioannidis. 2009. DBMSs Should Talk Back Too. arXiv:0909.1786 [cs.DB] https://arxiv.org/abs/0909.1786

[68] Sunanda Somwase, Parismita Das, and S. Sudarshan. 2025. Test Data Generation for Complex SQL Queries. *Proc. ACM Manag. Data* 3, 6, Article 367 (Dec. 2025), 26 pages. https://doi.org/10.1145/3769832

[69] Toni Taipalus and Piia Perälä. 2019. What to Expect and What to Focus on in SQL Query Teaching. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (Minneapolis, MN, USA) *(SIGCSE '19)*. Association for Computing Machinery, New York, NY, USA, 198–203. https://doi.org/10.1145/3287324.3287359

[70] Toni Taipalus, Mikko Siponen, and Tero Vartiainen. 2018. Errors and Complications in SQL Query Formulation. *ACM Trans. Comput. Educ.* 18, 3, Article 15 (Aug. 2018), 29 pages. https://doi.org/10.1145/3231712

[71] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) *(POPL '09)*. Association for Computing Machinery, New York, NY, USA, 264–276. https://doi.org/10.1145/1480881.1480915

[72] Yuan Tian, Jonathan K. Kummerfeld, Toby Jia-Jun Li, and Tianyi Zhang. 2024. SQLucid: Grounding Natural Language Database Queries with Interactive Explanations. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) *(UIST '24)*. Association for Computing Machinery, New York, NY, USA, Article 12, 20 pages. https://doi.org/10.1145/3654777.3676368

[73] Yuan Tian and Tianyi Zhang. 2026. PV-SQL: Synergizing Database Probing and Rule-based Verification for Text-to-SQL Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (Eds.). Association for Computational Linguistics, San Diego, California, United States, 25827–25845. https://doi.org/10.18653/v1/2026.findings-acl.1286

[74] Yuan Tian, Zheng Zhang, Zheng Ning, Toby Jia-Jun Li, Jonathan K. Kummerfeld, and Tianyi Zhang. 2023. Interactive Text-to-SQL Generation via Editable Step-by-Step Explanations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.).

Association for Computational Linguistics, Singapore, 16149–16166. https://doi.org/10.18653/v1/2023.emnlp-main.1004

[75] Etienne Toussaint, Paolo Guagliardo, Leonid Libkin, and Juan Sequeda. 2022. Troubles with nulls, views from the users. *Proc. VLDB Endow.* 15, 11 (July 2022), 2613–2625. https://doi.org/10.14778/3551793.3551818

[76] Boris A. Trakhtenbrot. 1950. Impossibility of an Algorithm for the Decision Problem in Finite Classes. *Doklady Akademii Nauk SSSR* 70 (1950), 569–572.

[77] Transaction Processing Performance Council. 2022. TPC Benchmark H (Decision Support): Standard Specification. https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-H_v3.0.1.pdf. Revision 3.0.1.

[78] Andrew Tremante, Yang He, Rocky Klopfenstein, Yuepeng Wang, Nina Narodytska, and Haoze Wu. 2026. SpotIt+: Verification-based Text-to-SQL Evaluation with Database Constraints. arXiv:2603.04334 [cs.DB] https://arxiv.org/abs/2603.04334

[79] Aditya Vashistha and Shrainik Jain. 2016. Measuring Query Complexity in SQLShare Workload. Unpublished manuscript. https://uwescience.github.io/sqlshare/pdfs/Jain-Vashistha.pdf

[80] Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. 2018. Robust Text-to-SQL Generation with Execution-Guided Decoding. arXiv:1807.03100 [cs.CL] https://arxiv.org/abs/1807.03100

[81] Shuxian Wang, Sicheng Pan, and Alvin Cheung. 2024. QED: A Powerful Query Equivalence Decider for SQL. *Proc. VLDB Endow.* 17, 11 (July 2024), 3602–3614. https://doi.org/10.14778/3681954.3682024

[82] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=1PL1NIMMrw

[83] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *Proc. VLDB Endow.* 13, 12 (July 2020), 1919–1932. https://doi.org/10.14778/3407790.3407799

[84] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. https://doi.org/10.1145/3434304

[85] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. 2025. Automated Validating and Fixing of Text-to-SQL Translation with Execution Consistency. *Proc. ACM Manag. Data* 3, 3, Article 134 (June 2025), 28 pages. https://doi.org/10.1145/3725271

[86] Ziyu Yao, Yu Su, Huan Sun, and Wen-tau Yih. 2019. Model-based Interactive Semantic Parsing: A Unified Framework and A Text-to-SQL Case Study. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 5447–5458. https://doi.org/10.18653/v1/D19-1547

[87] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. https://doi.org/10.18653/v1/D18-1425

[88] Yuxin Zhang, Meihao Fan, Ju Fan, Mingyang Yi, Yuyu Luo, Guoliang Li, Bin Wu, and Wenchao Zhou. 2026. Reward-SQL: Boosting Text-to-SQL via Stepwise Execution-Aware Reasoning and Process-Supervised Rewards. *Proc. ACM Manag. Data* 4, 3, Article 228 (May 2026), 27 pages. https://doi.org/10.1145/3802105

[89] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI, Article 125 (June 2023), 25 pages. https://doi.org/10.1145/3591239

[90] Fuheng Zhao, Jiayue Chen, Lawrence Lim, Ishtiyaque Ahmad, Divyakant Agrawal, and Amr El Abbadi. 2025. LLM-SQL-Solver: Can LLMs Determine SQL Equivalence?. In *2025 IEEE International Conference on Big Data (BigData)*. 1887–1894. https://doi.org/10.1109/BigData66926.2025.11401595

[91] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2020, 29 pages.

[92] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29

A PROOFS

A.1 Exactness without Shareable Classes

Theorem 6.1 (Exactness without shareable classes). *If no class is shareable, DP returns a decomposition in $\mathcal{D}_{\mathcal{R}}(q)$ of minimum J_θ.*

Proof outline. DP picks a block for every group bottom-up, combining the best choices for the inputs of a block (Equation (5)). This is the dynamic programming of memo-based plan enumeration [20, 63], and it is exact under the two conditions of the principle of optimality: the cost of a decomposition is the sum of the costs of its blocks, and the inputs of a block can be decomposed independently of each other. The first condition holds as no class is shareable, so that every block is a task of its own. The second is what needs proof in our setting: Lemma A.1 shows it, and Lemma A.2 then derives the recurrence of DP by the standard argument.

Scope. Definition 3.3 takes a finite, nonempty candidate set. As the memo is finite, so is $\mathcal{D}_{\mathcal{R}}(q)$; here and in Appendix A.2, we consider queries for which it is nonempty. (A query that only scans a table and selects columns has no block under rule (1) of Section 4.2.)

Notation. Let $\mathcal{H}$ consist of g_0 and the forced cuts, which DP solves separately (Section 6.1). For a block $b = (g, I)$, let $out(b) = g$, let $I^\circ(b) \subseteq I$ be its *non-forced inputs*, those that are not forced cuts, and let $w(b) = w_\theta(c)$ for its class c. A class is shareable if it contains two or more blocks of $\mathcal{B}$; every other class contains exactly one. For a set D of blocks, $out(D)$ and $C(D)$ denote the output groups and the classes of its blocks, and $\omega(D) = \sum_{b \in D} \omega(b)$ for a cost ω on blocks. An input of a block is the group of a proper subplan of a plan of its output, so it is a proper descendant of the output in the memo; as the memo is acyclic, we can define sets for groups and prove claims about them bottom-up, from descendants to ancestors.

Sub-decompositions. A *sub-decomposition* of a group p decomposes the part of q that p computes: it consists of a block b with output p and a sub-decomposition of each non-forced input of b. Bottom-up, the set $\mathcal{D}(p)$ of sub-decompositions of p is thus

$$\mathcal{D}(p) = \left\{ \{b\} \cup \bigcup_{i \in I^\circ(b)} D_i \;:\; b \in \mathcal{B},\ out(b) = p,\ D_i \in \mathcal{D}(i) \right\}.$$

Let $Out(p) = \bigcup_{D \in \mathcal{D}(p)} out(D)$ be the set of groups that receive a block in some sub-decomposition of p.

Lemma A.1 (Independence). (a) *For every block $b \in \mathcal{B}$, the sets $Out(i)$, $i \in I^\circ(b)$, are pairwise disjoint and do not contain $out(b)$, and the sets $Out(r)$, $r \in \mathcal{H}$, are pairwise disjoint.* (b) *Hence the blocks of every $D \in \mathcal{D}(p)$ have distinct output groups, and if $D = \{b\} \cup \bigcup_{i \in I^\circ(b)} D_i$, then $\omega(D) = \omega(b) + \sum_{i \in I^\circ(b)} \omega(D_i)$ for every cost ω.* (c) *$\mathcal{D}_{\mathcal{R}}(q)$ consists of the unions $\bigcup_{r \in \mathcal{H}} D_r$ with $D_r \in \mathcal{D}(r)$, whose sets D_r are pairwise disjoint.*

Part (a) says that the non-forced inputs of a block, and the groups of $\mathcal{H}$, can be decomposed separately: no block can serve two of them.

Proof. (a) We assign to each group the occurrences, that is, the FROM items (Section 4.1), that its plans read; all plans of a group read the same ones (Section 4.1). Two conventions complete this assignment. First, a `SELECT` without `FROM`, such as `SELECT 1 + 1`, reads a one-row source, which counts as an occurrence of its own; so two equal such subqueries at different positions are different groups. Second, a reference to a CTE that q reads from several places counts as an occurrence, but the occurrences in the CTE's body, which all its references share, do not; they count only for the groups within the body. This ignores no block, as no block reaches into such a body from outside: the body is a forced cut, which every block that reads it takes as an input (rule (3) of Section 4.2), unless it cannot be a block, and then no group in it outside the forced cuts that it reads is the output of a block (rule (1)). Three facts follow:

(i) every group reads at least one occurrence, at a leaf of its plans or as a reference above that leaf;
(ii) every occurrence counted for a plan lies at one position of it, so the non-forced inputs of b, which are disjoint parts of a plan of $out(b)$, read different occurrences;
(iii) a group in $Out(i)$ lies within the part of i, so it reads only occurrences that i reads.

By (i) and (iii), a group in two of the sets $Out(i)$ would read an occurrence that two inputs read, which (ii) excludes. Moreover, every group in $Out(i)$ is a proper descendant of $out(b)$, so none is $out(b)$. The same argument applies to g_0 and the forced cuts, which read each other only through references.

(b) By bottom-up induction, the blocks of each D_i in $D = \{b\} \cup \bigcup_i D_i$ have distinct output groups. These lie in $Out(i)$, and by (a), these sets are pairwise disjoint and do not contain $out(b)$. So all blocks of D have distinct output groups, the union is disjoint, and $\omega(D) = \omega(b) + \sum_i \omega(D_i)$.

(c) Every $D \in \mathcal{D}_{\mathcal{R}}(q)$ splits as stated. It has a block for g_0 and for every forced cut and, for each of its blocks, a block for every non-forced input, and no other blocks, as all its blocks are reachable from the root (Section 4.2). Following the non-forced inputs from the block of each $r \in \mathcal{H}$ thus collects a set $D_r \in \mathcal{D}(r)$, and these sets cover D and are disjoint by (a) and (b). Conversely, by (a) and (b), a union of sets $D_r \in \mathcal{D}(r)$, $r \in \mathcal{H}$, has exactly one block for each group that it decomposes, including every input of its blocks, and it satisfies the rules of Section 4.2, which restrict each block separately; so it lies in $\mathcal{D}_{\mathcal{R}}(q)$. □

As $\mathcal{B}$ consists of the blocks of these decompositions, $\mathcal{D}(out(b))$ and $\mathcal{D}(i)$, $i \in I^\circ(b)$, are nonempty for every $b \in \mathcal{B}$. The next lemma is the standard optimal-substructure argument of dynamic programming; we state it for any cost ω, as Appendix A.2 also applies it to other costs.

Lemma A.2 (Recurrence). *For a cost ω on blocks and a group p that is the output of a block of $\mathcal{B}$, let $best_\omega(p)$ be the least cost $\omega(D)$ of a sub-decomposition $D \in \mathcal{D}(p)$. Then*

$$best_\omega(p) = \min_{b \in \mathcal{B},\ out(b) = p} \left[\omega(b) + \sum_{i \in I^\circ(b)} best_\omega(i) \right],$$

which is Equation (5) for the cost ω with the forced cuts left out (Section 6.1). Choosing a minimizing block at p and, recursively, at its non-forced inputs yields a sub-decomposition $D^*_\omega(p) \in \mathcal{D}(p)$ of cost $best_\omega(p)$.

Proof. A sub-decomposition of p consists of a block b with output p and sub-decompositions $D_i \in \mathcal{D}(i)$ of its non-forced inputs, chosen independently of each other, and its cost is $\omega(b) + \sum_i \omega(D_i)$ by Lemma A.1(b). For a fixed b, choosing each D_i of least

cost minimizes this sum, to $\omega(b) + \sum_i best_\omega(i)$; minimizing over b gives the recurrence, and these choices form $D^*_\omega(p)$. □

Proof of Theorem 6.1. As no class is shareable, distinct blocks have distinct classes, so $J_\theta(D) = w(D)$ for every set D of blocks. The sets $\mathcal{D}(r)$, $r \in \mathcal{H}$, are nonempty, as $\mathcal{D}_\mathcal{R}(q)$ is, and DP returns $D^* = \bigcup_{r\in\mathcal{H}} D^*_w(r)$ (Lemma A.2), which lies in $\mathcal{D}_\mathcal{R}(q)$ by Lemma A.1(c). By the same lemma, every $D \in \mathcal{D}_\mathcal{R}(q)$ is the disjoint union of sets $D_r \in \mathcal{D}(r)$, $r \in \mathcal{H}$, and so is D^*, of the sets $D^*_w(r)$; hence

$$J_\theta(D) = \textstyle\sum_{r\in\mathcal{H}} w(D_r) \geq \sum_{r\in\mathcal{H}} best_w(r) = J_\theta(D^*).$$ □

A.2 Exactness of the Search

Theorem 6.2 (Exactness of the search). *The heuristic h is admissible and consistent, and Algorithm 1 returns a decomposition in $\mathcal{D}_\mathcal{R}(q)$ of minimum J_θ.*

Proof outline. A decomposition can share work in two ways. A group read from several places, the body of a CTE that q reads more than once, is a forced cut, which DP solves once (Appendix A.1). Different blocks can share one task, by belonging to one shareable class; this couples their costs, and the search therefore records the shareable classes paid so far in Φ. Without shareable classes, Algorithm 1 returns the result of DP, which is optimal by Theorem 6.1. Otherwise, it runs A* on a graph of search states, and A* returns a cheapest path to a goal when its heuristic is consistent (Lemma A.6). Three facts about the search then give the theorem: every goal yields a decomposition whose cost J_θ is what the search paid to reach it (Lemma A.3); Fold preserves the optimal cost, as every decomposition has a reachable goal that is at least as cheap (Lemma A.4); and h is consistent (Lemma A.5).

Notation. We keep the scope, the notation, and the results of Appendix A.1. Every class cost is nonnegative, as $\mu \geq 0$, $R(\theta, c) \geq 0$, $\gamma \geq 0$, and $s(c) \in (0, 1]$. A group p *can use* a class if the class occurs in some sub-decomposition of p. If a block with output p has a non-forced input i, every sub-decomposition of i is contained in some sub-decomposition of p; so $Out(i) \subseteq Out(p)$, and i can use only classes that p can use. For a set $\Phi \subseteq C_{sh}$ of paid classes, w_Φ is the cost w with the classes of Φ free: $w_\Phi(b) = 0$ if the class of b lies in Φ, and $w_\Phi(b) = w(b)$ otherwise. The heuristic makes every shareable class free: with $\hat{w} = w_{C_{sh}}$, it is $h(\sigma) = \sum_{p\in\Pi} best_{\hat{w}}(p)$ for a state $\sigma = (\Pi, \Phi)$.

The search graph. A state $\sigma = (\Pi, \Phi)$ holds the pending groups Π and the paid shareable classes Φ; along with it, Algorithm 1 keeps the set D_σ of blocks chosen so far and the part $J(\sigma)$ of J_θ paid so far. Three kinds of steps change them:

(M) A *move* gives the first pending group p a block b with output p. It pays $w_\Phi(b)$, adds b to D_σ, replaces p by the non-forced inputs of b in Π, and adds the class of b to Φ if that class is shareable.

(F1) Fold solves a pending group p that can use no shareable class outside Φ. It pays $best_{w_\Phi}(p)$, adds $D^*_{w_\Phi}(p)$ to D_σ, and removes p from Π.

(F2) Fold drops from Φ the classes that no pending group can use.

The search begins at $(\mathcal{H}, \emptyset)$ with $D_\sigma = \emptyset$ and $J(\sigma) = 0$, and Fold turns this into the start state. An edge from σ is a move followed by Fold, and its cost is the amount they pay; the edges of a state and their costs depend only on (Π, Φ). The goals are the states with $\Pi = \emptyset$. Algorithm 1 runs A* on this graph and keeps only the cheapest path found to each state (Section 6.2): line 9 relaxes Fold(σ'), that is, the state enters Q, with $D_{\sigma'}$ and $J(\sigma')$, only if no path to it found before is as cheap, and it then replaces the entry of such a path.

Step (F1). It is the DP of Appendix A.1 applied to p, with the classes of Φ free, and it is exact for the same reason as Theorem 6.1. As p can use no shareable class outside Φ, every class of a sub-decomposition $D_p \in \mathcal{D}(p)$ outside Φ is not shareable and has its only block in D_p; so

$$w_\Phi(D_p) = \textstyle\sum_{c\in C(D_p)\setminus\Phi} w_\theta(c), \qquad (*)$$

the cost of the classes of D_p outside Φ, and (F1) adds a cheapest such D_p (Lemma A.2).

Lemma A.3 (Invariants). *After every sequence of steps from $(\mathcal{H}, \emptyset)$:* (a) *the chosen blocks and the pending parts do not overlap: the sets $out(D_\sigma)$ and $Out(p)$, $p \in \Pi$, are pairwise disjoint, and the blocks of D_σ have distinct output groups;* (b) *Φ records the paid shareable classes that may still be used: its classes are classes of D_σ, and it contains every shareable class of D_σ that a pending group can use;* (c) *$J(\sigma)$ is the cost of the chosen blocks, each class counted once: $J(\sigma) = \sum_{c\in C(D_\sigma)} w_\theta(c)$;* (d) *every completion is a decomposition: $D_\sigma \cup \bigcup_{p\in\Pi} D_p \in \mathcal{D}_\mathcal{R}(q)$ for every choice of $D_p \in \mathcal{D}(p)$, $p \in \Pi$. In particular, at a goal σ, $D_\sigma \in \mathcal{D}_\mathcal{R}(q)$ and $J(\sigma) = J_\theta(D_\sigma)$.*

Proof. By induction on the number of steps. Initially, $D_\sigma = \Phi = \emptyset$, $J(\sigma) = 0$, and $\Pi = \mathcal{H}$, so (a) holds by Lemma A.1(a), (b) and (c) hold trivially, and (d) is Lemma A.1(c). We show that every step preserves (a)–(d).

Structure: (a) and (d). A move replaces a pending group p by the block b given to it and the non-forced inputs i of b; step (F1) replaces p by the blocks of $D^* = D^*_{w_\Phi}(p) \in \mathcal{D}(p)$. In both cases, the new blocks and pending groups lie within $Out(p)$, which was disjoint from $out(D_\sigma)$ and from the sets $Out(p')$ of the other pending groups p'; within it, p and the sets $Out(i)$ are pairwise disjoint, and the output groups of D^* are distinct, by Lemma A.1(a,b). So (a) still holds. For (d), a completion after the step is one before it, with $D_p = \{b\} \cup \bigcup_i D_i$ or $D_p = D^*$. Step (F2) changes neither D_σ nor Π.

Record: (b). A move or (F1) only replaces a pending group by its inputs or removes it, so the pending groups can use no new class. A move adds the class of its block to the classes of D_σ, and to Φ if the class is shareable; the shareable classes of D^* in (F1) can be used by p, so they already lie in Φ; and (F2) drops only classes that no pending group can use. So (b) still holds.

Charge: (c). Let a move give p a block b of class c. It pays $w_\theta(c)$ if $c \notin \Phi$ and nothing otherwise, so it suffices that $c \in \Phi$ exactly when c is already a class of D_σ. If $c \in \Phi$, it is one by (b). Conversely, if c is a class of D_σ, it is shareable, as otherwise its only block b would lie in D_σ, and $p \in out(D_\sigma) \cap Out(p)$ would contradict (a); and p can use c, so $c \in \Phi$ by (b). Step (F1) pays $w_\Phi(D^*)$, which by $(*)$ is the cost of the classes of D^* outside Φ; the classes in Φ are already paid, and the others are new, as their only blocks lie in D^*, whose output groups are not in $out(D_\sigma)$ by (a). Step (F2) pays nothing and changes no class of D_σ.

At a goal, $\Pi = \emptyset$, so (d) gives $D_\sigma \in \mathcal{D}_\mathcal{R}(q)$, and (c) gives $J(\sigma) = J_\theta(D_\sigma)$. □

Lemma A.4 (Optimality preservation). *For every $D \in \mathcal{D}_{\mathcal{R}}(q)$, the search graph has a path from the start state to a goal σ with $J(\sigma) \le J_\theta(D)$.*

So Fold may discard decompositions, but never the optimal cost: every decomposition has a reachable goal of no greater cost.

Proof. Fix $D \in \mathcal{D}_{\mathcal{R}}(q)$. Along the steps from $(\mathcal{H}, \emptyset)$, we keep a decomposition

$$E = D_\sigma \cup \bigcup_{p \in \Pi} E_p, \qquad E_p \in \mathcal{D}(p),$$

that completes the current state, with $J_\theta(E) \le J_\theta(D)$. Initially, $E = D$, split as in Lemma A.1(c). Each move gives the first pending group p the block b of E_p with output p; then $E_p = \{b\} \cup \bigcup_i E_i$ with $E_i \in \mathcal{D}(i)$ for the non-forced inputs i of b, and E does not change. Step (F2) does not change E either. Step (F1) replaces E_p by $D^* = D^*_{w_\Phi}(p)$, and the result E' is again a decomposition, by Lemma A.3(d). Only the classes of E_p and D^* outside Φ can change the cost: those in Φ are classes of D_σ by Lemma A.3(b), and the others are not shareable and have their only blocks in E_p or D^*, whose output groups lie in $Out(p)$ and so, by Lemma A.3(a), nowhere else in E or E'. Hence, by $(*)$ and Lemma A.2,

$$J_\theta(E') - J_\theta(E) = w_\Phi(D^*) - w_\Phi(E_p) \le 0.$$

Each move or (F1) step replaces a pending group by groups below it or removes it, so, as the memo is finite and acyclic, these steps reach a goal σ, where $E = D_\sigma$. By Lemma A.3, $J(\sigma) = J_\theta(D_\sigma) = J_\theta(E) \le J_\theta(D)$. □

Lemma A.5 (Heuristic). *For every edge from σ to σ' with cost k between states reached from the start state, $h(\sigma) \le k + h(\sigma')$, and $h(\sigma) = 0$ at every goal; that is, h is consistent. Hence h is admissible: $h(\sigma)$ is at most the cost of every path from σ to a goal.*

Proof. At a goal, $\Pi = \emptyset$, so $h(\sigma) = 0$. We show that no step of an edge decreases h by more than it pays; adding up over the steps of the edge then gives $h(\sigma) \le k + h(\sigma')$. *(M)* The move gives the first pending group p a block b and pays $w_\Phi(b)$. As $\hat{w}$ makes every shareable class free, $\hat{w}(b) \le w_\Phi(b)$, and so, by Lemma A.2,

$$best_{\hat{w}}(p) \le w_\Phi(b) + \textstyle\sum_{i \in I^\circ(b)} best_{\hat{w}}(i).$$

The move replaces the term $best_{\hat{w}}(p)$ of h by the terms $best_{\hat{w}}(i)$, $i \in I^\circ(b)$, whose groups are distinct from each other and from the other pending groups by Lemma A.3(a), so h decreases by at most $w_\Phi(b)$. *(F1)* Solving a group p pays $best_{w_\Phi}(p)$ and removes the term $best_{\hat{w}}(p)$ from h. The two are equal: both are minima over $\mathcal{D}(p)$, and w_Φ and $\hat{w}$ agree on the blocks of its members, as p can use no shareable class outside Φ. *(F2)* Neither h nor the cost changes. Summing $h(\sigma) \le k + h(\sigma')$ along a path from σ to a goal, where $h = 0$, shows that h is admissible. □

Lemma A.6 (A*). *Let a finite graph have nonnegative edge costs and a goal reachable from its start, and let h be consistent and 0 at goals. Then A*, which keeps only the cheapest path found to each state, expands every state at most once and returns a cheapest path from the start to a goal.*

This is the standard optimality of A* with a consistent heuristic [23]; we include a short proof.

Proof. For an edge from u to v of cost $k(u, v)$, let $k_h(u, v) = k(u, v) + h(v) - h(u)$, which is nonnegative as h is consistent. A path from the start to v of cost ℓ costs $\ell + h(v) - h(start)$ under k_h, so A*, which takes from the queue a state of least $\ell + h$, is Dijkstra's algorithm on the graph with the costs k_h. With nonnegative costs, Dijkstra's algorithm takes every state from the queue at most once, with a cheapest path to it, in the order of the costs of these paths. As $h = 0$ at goals, every path to a goal costs $h(start)$ less under k_h than under k, so the first goal taken from the queue is reached by a path of least cost among all paths from the start to a goal. □

As $\Phi \subseteq C_{sh}$ in every state, A* thus expands at most $2^{|C_{sh}|}$ states for each set of pending groups, as Proposition 6.3 states.

Proof of Theorem 6.2. By Lemma A.5, h is consistent and admissible. If no class is shareable, Algorithm 1 returns the result of DP, which is optimal by Theorem 6.1. Otherwise, every goal σ reached from the start state yields a decomposition $D_\sigma \in \mathcal{D}_{\mathcal{R}}(q)$ with $J_\theta(D_\sigma) = J(\sigma)$ (Lemma A.3), and every $D \in \mathcal{D}_{\mathcal{R}}(q)$ has such a goal with $J(\sigma) \le J_\theta(D)$ (Lemma A.4). Hence

$$\min_\sigma J(\sigma) = \min_{D \in \mathcal{D}_{\mathcal{R}}(q)} J_\theta(D),$$

where σ ranges over the goals reached from the start state; both minima exist, as $\mathcal{D}_{\mathcal{R}}(q) \neq \emptyset$. The search graph is finite, as Π is a set of groups and Φ a set of classes, and its edge costs are nonnegative. Along a path from the start state, J grows by the cost of the path, so by Lemma A.6, A* returns a goal σ_G that attains the left-hand minimum, and D_{σ_G} attains the right-hand one. □

B PROMPTS

We list the prompts of the two semantic measures (Section 5) and of the validator. Each is a template in which a name in braces, such as `{question}`, is filled in for every call. The other prompts are in the `prompts` folder of our artifact repository.[1]

B.1 Coherence

The relations are the cut points to rate, each written as one SQL statement over the base tables (Section 7). For a relation inside a correlated subquery, `{closed_form}` adds an explanation of how it is written.

```
Below are intermediate relations computed inside a SQL query, each written
as a stand-alone SQL statement over the base tables.{closed_form} You are
not told what the query is for.

For each relation, judge whether it makes sense on its own: whether a person
who knows the schema could understand it without seeing the rest of the
query.

First write "name": a short noun phrase (at most 8 words) for what the
relation is, and "row": what one row of it is (for example "one row per
customer"). Then rate "coherence" from 0 to 3:
3 = a natural unit with a clear name and a clear row (for example "customers
in Europe"; "number of orders per customer").
2 = makes sense, but the name needs a qualification, or some columns are
there only for a later step.
1 = describable only as an operation ("rows of A joined with B"): a staging
step whose purpose would show only later, or an unclear row (one row per
what?).
0 = does not make sense on its own: for example half of a condition, or
columns whose meaning depends on something not shown.

Judge the meaning only: not whether the SQL is correct, and not its length
or complexity. A small relation can get 3. Do not guess what the query is
for.
```

[1] https://github.com/Inverse0/validation-aware-decomposition

```
SCHEMA
{schema_text}

RELATIONS
{relations}

Answer with ONE JSON object and nothing else: {"C1": {"name": "...", "row":
"...", "coherence": n}, "C2": {...}, ...} with exactly the labels of the
relations above as keys, every one of them.
```

B.2 Relevance

The relations and {closed_form} are as for coherence; {evidence} holds the additional information given with the question, if any.

```
A SQL query answers the natural-language question below. The relations below
the question are computed inside that query, each written as a stand-alone
SQL statement over the base tables.{closed_form}

For each relation, judge whether it is a concept the question mentions. Give
"match", one of:
"complete" = it computes a concept the question names or asks for (an entity
set, a quantity, the answer itself) with every condition the question
attaches to that concept.
"narrower" = it computes such a concept with some of the conditions the
question attaches to it applied in full and the others still to come: a
natural step toward that concept (for example, the question asks about toy
orders placed in 2020 and the relation is the orders placed in 2020).
"split" = one condition of the question is applied only in part: one end of
a range the question gives, or a match between two things on only some of
the attributes the question matches them on (for example, the question pairs
each order with a payment by the same customer on the same day, and the
relation pairs orders with payments by the same customer on any day), so
that its rows only make sense once a later step applies the rest of that
condition. A relation that applies some whole conditions and leaves the
others to a later step is narrower, not split, also when it joins tables to
apply them (for example, the question asks about customers who placed an
order and wrote a review, and the relation is the customers who placed an
order).
"technical" = it matches no phrase of the question and splits none: a
preparatory step (bringing tables together, a lookup).
"unrelated" = no connection to the question.
Also give "phrase": the words of the question that the relation matches ("" 
for technical or unrelated), and "missing": for narrower or split, what the
question says that is not applied, or applied only in part, in the relation
("" otherwise).

Only what the question says about which rows belong counts (its conditions,
including a threshold or a top-k): a computation still to come that does not
decide which rows belong (an aggregate over them, the columns shown, a total
row) does not make a relation narrower. A small relation that matches one
phrase of the question is "complete", exactly like the full answer: being
closer to the full answer earns nothing. Do not judge whether the SQL is
correct.

QUESTION
{question}
{evidence}
SCHEMA
{schema_text}

RELATIONS
{relations}

Answer with ONE JSON object and nothing else: {"C1": {"match": "...",
"phrase": "...", "missing": "..."}, "C2": {...}, ...} with exactly the
labels of the relations above as keys, every one of them.
```

B.3 Validation

DecoVal, Whole, and Whole $\times k$ use the same prompt. For a whole query, {question} is the question and {inputs_text} is empty. For a block, {question} is its requirement followed by judging instructions, and {inputs_text} describes its inputs with their established facts (Section 7). {schema_legend} explains the notation of the schema.

```
You are reviewing a SQL query written to answer a question. Decide whether
the query, exactly as written, answers the question correctly against the
schema below. Judge the SQL text itself: its joins, filters, grouping,
output columns and the semantics of its operators. Do not rewrite it and do
not run anything. Minor formatting differences, column aliases and row order
do not matter unless the question asks for an order.

QUESTION
{question}
{evidence}
{schema_legend}
{schema_text}
{inputs_text}
SQL
{sql}

Answer with exactly two lines and nothing else:
VERDICT: yes
REASON: <one sentence>
or
VERDICT: no
REASON: <one sentence>
```

C USER STUDY MATERIALS

Both questions come from the dw split of BEAVER [9] and read three of its tables. We renamed the columns to self-explanatory names, so that participants could focus on the logic of the queries. We also made each question precise, following BEAVER's gold SQL, so that the question alone determines whether a candidate is correct.

Task 1 (dw_4704). "For each department in the School of Engineering with a total number of students enrolled in its courses (over all terms) greater than 50, list the department name, the total number of enrolled students, and for the term 2019FA, the subject with the highest enrollment offered by that department (if several subjects tie, list each of them), including the subject title and the number of enrolled students. Present the results ordered by department name. Do not return any rounded answers." Additional information: "The School of Engineering is school_name = 'Engineering'." The correct candidate is:

```sql
SELECT
  totals.department_name,
  totals.total_enrollment,
  so1.subject_title AS highest_enrolled_subject,
  so1.num_enrolled_students AS highest_enrollment
FROM (
  SELECT
    dep.department_code,
    dep.department_name,
    SUM(so.num_enrolled_students) AS total_enrollment
  FROM department AS dep
  JOIN subject_offering AS so
    ON so.offering_department_code = dep.department_code
  WHERE
    dep.school_name = 'Engineering'
  GROUP BY
    dep.department_code,
    dep.department_name
) AS totals
JOIN subject_offering AS so1
  ON so1.offering_department_code = totals.department_code
JOIN (
  SELECT
    so2.offering_department_code,
    MAX(so2.num_enrolled_students) AS max_enrollment
  FROM subject_offering AS so2
  WHERE
    so2.term_code = '2019FA'
  GROUP BY
    so2.offering_department_code
) AS dept_max
  ON dept_max.offering_department_code = so1.offering_department_code
  AND so1.num_enrolled_students = dept_max.max_enrollment
WHERE
  totals.total_enrollment > 50 AND so1.term_code = '2019FA'
ORDER BY
  totals.department_name
```

Task 2 (dw_3016). "For subjects offered in the term 2019FA, show the subject ID, subject title, number of enrolled students, department name, department full name, and indicate whether the number of enrolled students is 'HIGH_ENROLLMENT' (more than 10% above the average for that department), 'LOW_ENROLLMENT' (more than 10% below the average for that department), or 'AVERAGE', along with the department's average enrollment. A department's average is taken over its subjects offered in 2019FA. Do not return any rounded answers." The correct candidate is:

```
SELECT
  so.subject_id,
  so.subject_title,
  so.num_enrolled_students,
  dep.department_name,
  dep.department_full_name,
  dept_avg.avg_enrollment,
  CASE
    WHEN so.num_enrolled_students > dept_avg.avg_enrollment * 1.1
    THEN 'HIGH_ENROLLMENT'
    WHEN so.num_enrolled_students < dept_avg.avg_enrollment * 0.9
    THEN 'LOW_ENROLLMENT'
    ELSE 'AVERAGE'
  END AS enrollment_category
FROM subject_offering AS so
JOIN department AS dep
  ON so.offering_department_code = dep.department_code
JOIN (
  SELECT
    dep2.department_name,
    AVG(so2.num_enrolled_students) AS avg_enrollment
  FROM subject_offering AS so2
  JOIN department AS dep2
    ON so2.offering_department_code = dep2.department_code
  WHERE
    so2.term_code = '2019FA'
  GROUP BY
    dep2.department_name
) AS dept_avg
  ON dep.department_name = dept_avg.department_name
WHERE
  so.term_code = '2019FA'
```

Incorrect candidates. Each task has two incorrect candidates; Table 6 gives the error of each.

Table 6: The errors of the incorrect candidates.

Task	Error	Description
1	A	The correlated subquery that takes the department's highest enrollment lacks `so2.term_code = '2019FA'`, so it takes the maximum over all terms: a department whose maximum is reached only in another term is missing.
	B	`ORDER BY so2.num_enrolled_students DESC LIMIT 1` keeps one subject per department: of several 2019FA subjects tied for the highest enrollment, only one is listed.
2	A	The `CASE` tests AVERAGE (`>= dept_avg.avg_enrollment * 0.9`) before HIGH, so no subject is `'HIGH_ENROLLMENT'`.
	B	`SUM(so2.num_enrolled_students) / COUNT(*)` divides integers, so the department's average is rounded down.

D MEASURES

Let N_{inc} and N_{cor} be the numbers of incorrect and correct candidates (243 and 62), and $N = N_{inc} + N_{cor}$. An incorrect candidate without a verdict counts in FN, and a correct one in neither TN nor FP. The measures are

$$\mathrm{Acc} = \frac{TP + TN}{N}, \qquad \mathrm{Prec} = \frac{TP}{TP + FP},$$
$$\mathrm{Rec} = \frac{TP}{N_{inc}}, \qquad \mathrm{Spec} = \frac{TN}{N_{cor}},$$
$$F_1 = \frac{2\,\mathrm{Prec} \cdot \mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}}, \qquad \mathrm{BA} = \frac{\mathrm{Rec} + \mathrm{Spec}}{2}.$$